

操作系统课程设计
项目说明文档

MIT Xv6 2021 Labs

学 号 : 2453203

姓 名 : 李欣鸿

班 级 : 42028704

学 院 专 业 : 计算机科学与技术学院

软件工程专业



同济大学

Tongji University

目录

实验准备	8
一、环境搭建	8
1. 安装 Linux 的 Windows 子系统	8
2. 安装所需的软件或依赖:	9
二、下载实验代码	10
Lab1: Xv6 and Unix utilities	12
Boot xv6 (easy)	12
sleep (easy)	15
一、实验目的	15
二、实验步骤	15
三、实验中遇到的问题和解决办法	17
四、实验心得	17
pingpong (easy)	17
一、实验目的	17
二、实验步骤	17
三、实验中遇到的问题和解决办法	20
四、实验心得	20
primes (moderate)/(hard)	20
一、实验目的	20
二、实验步骤	20
三、实验中遇到的问题和解决办法	24

四、实验心得	24
find (moderate)	24
一、实验目的	24
二、实验步骤	24
三、实验中遇到的问题和解决办法	28
四、实验心得	28
xargs (moderate)	29
一、实验目的	29
二、实验步骤	29
三、实验中遇到的问题和解决办法	31
四、实验心得	32
Lab2: system calls	32
System call tracing (moderate)	32
一、实验目的	32
二、实验步骤	33
三、实验中遇到的问题和解决办法	43
四、实验心得	43
Sysinfo (moderate)	44
一、实验目的	44
二、实验步骤	44
三、实验中遇到的问题和解决办法	48
四、实验心得	48

Lab3: page tables	49
Speed up system calls (easy)	49
一、实验目的	49
二、实验步骤	50
三、实验中遇到的问题和解决办法	51
四、实验心得	52
Print a page table (easy)	53
一、实验目的	53
二、实验步骤	53
三、实验中遇到的问题和解决办法	55
四、实验心得	55
Detecting which pages have been accessed (hard)	56
一、实验目的	56
二、实验步骤	56
三、实验中遇到的问题和解决办法	58
四、实验心得	58
Lab4: traps	59
RISC-V assembly (easy)	59
Q1:	59
Q2:	60
Q3:	61
Q4:	62

Q5:.....	63
Q6:.....	67
Backtrace (moderate)	68
一、实验目的	68
二、实验步骤	68
三、实验中遇到的问题和解决办法	71
四、实验心得	71
Alarm (hard)	72
一、实验目的	72
二、实验步骤	72
三、实验中遇到的问题和解决办法	79
四、实验心得	79
Lab5: Copy-on-Write Fork for xv6	80
Implement copy-on write(hard)	80
一、实验目的	80
二、实验步骤	81
三、实验中遇到的问题和解决办法	86
四、实验心得	86
Lab6: Multithreading	87
Uthread: switching between threads (moderate)	87
一、实验目的	87
二、实验步骤	87

三、实验遇到的问题和解决办法.....	90
四、实验心得.....	90
Using threads (moderate).....	90
一、实验目的.....	90
二、实验步骤.....	90
三、实验遇到的问题和解决办法.....	92
四、实验心得.....	92
Barrier(moderate).....	93
一、实验目的.....	93
二、实验步骤.....	93
三、实验遇到的问题和解决办法.....	95
四、实验心得.....	95
Lab7: networking.....	95
Your Job (hard).....	96
一、实验目的.....	96
二、实验步骤.....	96
三、实验中遇到的问题和解决办法.....	100
四、实验心得.....	100
Lab8: locks.....	100
Memory allocator (moderate).....	101
一、实验目的.....	101
二、实验步骤.....	101

三、实验中遇到的问题和解决办法	105
四、实验心得	106
Buffer cache (hard)	106
一、实验目的	106
二、实验步骤	106
三、实验中遇到的问题和解决办法	112
四、实验心得	113
Lab9: file system	113
Large files (moderate)	113
一、实验目的	113
二、实验步骤	114
三、实验遇到的问题和解决办法	116
四、实验心得	116
Symbolic links (moderate)	117
一、实验目的	117
二、实验步骤	117
三、实验遇到的问题与解决方法	120
四、实验心得	120
Lab10: mmap	121
mmap(hard)	121
一、实验目的	121
二、实验步骤	121

三、实验中遇到的问题和解决办法.....	135
四、实验心得.....	136

源代码: <https://github.com/Comeat0205/Tongji-2026-Summer-OS-xv6-Labs.git>

实验准备

一、环境搭建

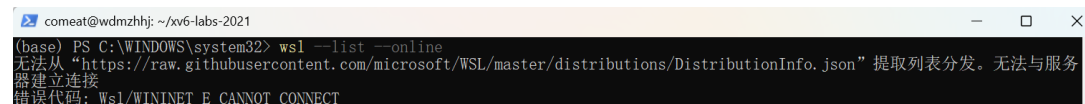
1. 安装 Linux 的 Windows 子系统

PowerShell

```
wsl --install -d Ubuntu-20.04
```

注:

1. 我感觉这里的 Ubuntu 版本还是蛮重要的, 第一次下载的是 Ubuntu 26.04 , 中间尝试一直出问题, 但是尝试下载了 20.04 的版本就很顺利, 而且我觉得不用从 Microsoft Store 中添加, 就执行上面的命令就行
2. 建议安装的时候把梯子关掉 (不太确定是不是梯子的影响, 但是关掉之前确实不能下载)



The screenshot shows a PowerShell terminal window with the title bar 'comeat@wdmzhj: ~/xv6-labs-2021'. The command '(base) PS C:\WINDOWS\system32> wsl --list --online' has been executed. The output is an error message in Chinese: '无法从 "https://raw.githubusercontent.com/microsoft/WSL/master/distributions/DistributionInfo.json" 提取列表分发。无法与服务建立连接' (Unable to extract the list of distributions from "https://raw.githubusercontent.com/microsoft/WSL/master/distributions/DistributionInfo.json". Unable to establish connection with the service). Below this, the error code is shown: '错误代码: Wsl/WSLINET_E_CANNOT_CONNECT'.

更具体信息可参考: [Linux 的 Windows 子系统](#)

更改安装的默认Linux发行版

默认情况下，安装的Linux发行版将是Ubuntu。这可以通过旗帜进行更改。 `-d`

- 要更改已安装的发行版，请输入：

PowerShell 收到

```
wsl.exe --install -d [Distro]
```

请用你想安装的发行版名称替换。 `[Distro]`

- 要查看在线商店可下载的Linux发行版列表，请输入：

PowerShell 收到

```
wsl.exe --list --online
```

安装成功截图如下，需要创建用户账号和密码：

```
comeat@wdmzhj: ~/xv6-labs-2021
(base) PS C:\WINDOWS\system32> wsl --install -d Ubuntu-20.04
正在下载: Ubuntu 20.04 LTS
正在安装: Ubuntu 20.04 LTS
已成功安装分发。可以通过 “wsl.exe -d Ubuntu-20.04” 启动它
正在启动 Ubuntu-20.04...
Provisioning the new WSL instance Ubuntu-20.04
This might take a while...
Create a default Unix user account: comeat
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

2. 安装所需的软件或依赖：

PowerShell

安装相关依赖

```
$ sudo apt-get update && sudo apt-get upgrade
```

```
$ sudo apt-get install git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

```

comeat@wdmzhhj: ~/xv6-labs-2021
comeat@wdmzhhj:~/xv6-labs-2021$ sudo apt-get update && sudo apt-get upgrade
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Hit:2 http://archive.ubuntu.com/ubuntu focal InRelease
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [3610 kB]
Get:5 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal/universe amd64 Packages [8628 kB]
Get:7 http://security.ubuntu.com/ubuntu focal-security/main Translation-en [518 kB]
Get:8 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 Packages [3828 kB]
Get:9 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages [1052 kB]
Get:10 http://security.ubuntu.com/ubuntu focal-security/universe Translation-en [220 kB]
Get:11 http://security.ubuntu.com/ubuntu focal-security/universe amd64 c-n-f Metadata [22.4 kB]
Get:12 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 Packages [26.7 kB]
Get:13 http://security.ubuntu.com/ubuntu focal-security/multiverse Translation-en [6416 B]
Get:14 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 c-n-f Metadata [604 B]
Get:15 http://archive.ubuntu.com/ubuntu focal/universe Translation-en [5124 kB]
Get:16 http://archive.ubuntu.com/ubuntu focal/universe amd64 c-n-f Metadata [265 kB]
Get:17 http://archive.ubuntu.com/ubuntu focal/multiverse amd64 Packages [144 kB]
Get:18 http://archive.ubuntu.com/ubuntu focal/multiverse Translation-en [104 kB]
Get:19 http://archive.ubuntu.com/ubuntu focal/multiverse amd64 c-n-f Metadata [9136 B]
Get:20 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [4004 kB]
Get:21 http://archive.ubuntu.com/ubuntu focal-updates/main Translation-en [601 kB]
Get:22 http://archive.ubuntu.com/ubuntu focal-updates/restricted amd64 Packages [3984 kB]
Get:23 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [1275 kB]
Get:24 http://archive.ubuntu.com/ubuntu focal-updates/universe Translation-en [302 kB]
Get:25 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 c-n-f Metadata [29.3 kB]
Get:26 http://archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 Packages [29.8 kB]
Get:27 http://archive.ubuntu.com/ubuntu focal-updates/multiverse Translation-en [8284 B]
Get:28 http://archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 c-n-f Metadata [688 B]
Get:29 http://archive.ubuntu.com/ubuntu focal-backports/main amd64 Packages [46.1 kB]

comeat@wdmzhhj: ~/xv6-labs-2021
Preparing to unpack .../distro-info-data_0.43ubuntu1.20_all.deb ...
Unpacking distro-info-data (0.43ubuntu1.20) over (0.43ubuntu1.19) ...
Setting up distro-info-data (0.43ubuntu1.20) ...
comeat@wdmzhhj:~/xv6-labs-2021$ sudo apt-get install git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu
Reading package lists... Done
Building dependency tree
Reading state information... Done
git is already the newest version (1:2.25.1-1ubuntu3.14).
git set to manually installed.
The following additional packages will be installed:
acl cpp cpp-9 cpp-9-riscv64-linux-gnu cpp-riscv64-linux-gnu dpkg-dev fakeroot g++ g++-9 gcc gcc-10-cross-base-ports
gcc-9 gcc-9-base gcc-9-cross-base-ports gcc-9-riscv64-linux-gnu gcc-9-riscv64-linux-gnu-base gdb gdbserver
gststreamer1.0-plugins-base gststreamer1.0-plugins-good gststreamer1.0-x-ibverbs-providers ipxe-qemu libaal libaio
libalgoritm-diff-perl libalgoritm-diff-xs-perl libalgoritm-merge-perl libasan5 libatomic1 libatomic1-riscv64-cross
libavc1394-0 libbabeltrace1 libboost-iostreams1.71.0 libboost-thread1.71.0 libbrlapi0.7 libc-dev-bin libc6-dbg libc6-dev
libc6-dev-riscv64-cross libc6-riscv64-cross libcaca0 libcacard0 libcc1-0 libcdparanoia0 libcrypt-dev libdpkg-perl libdv4
libdw1 libfakeroot libfdt1 libfile-fcntllock-perl libgcc-9-dev libgcc-9-dev-riscv64-cross libgcc-s1-riscv64-cross
libgcompl1 libgcompl1-riscv64-cross libgstreamer-plugins-base1.0-0 libgstreamer-plugins-good1.0-0 libgudev-1.0-0 libibverbs1
libiec61883-0 libiscsi7 libis122 libisl12 libltl libjack-jackd2-0 liblsan0 libmp3lame0 libmpc3 libmpg123-0 libnl-3-200
libnl-route-3-200 libnspr4 libnss3 libopus0 liborc-0.4-0 libpcsclite1 libpnm1 libquadmath0 librados2 libraw1394-11
librbd1 librdmacm1 libsamplero0 libshout3 libslirp0 libspeex1 libspice-server1 libstdc++-9-dev libtag15
libtag15-vanilla libtheora0 libtsan0 libtwolame0 libubsan1 libusbredirparser1 libv4l-0 libv4lconvert0 libvirglrenderer1
libvisual-0.4-0 libvpx6 libvte-2.91-0 libvte-2.91-common libwvpack1 linux-libc-dev linux-libc-dev-riscv64-cross
manpages-dev qemu-block-extra qemu-system-common qemu-system-data qemu-system-gui qemu-utils seabios sharutils
Suggested packages:
binutils-cross cpp-doc gcc-9-locales debian-keyring g++-multilib g++-9-multilib gcc-9-doc gcc-multilib autoconf automake
libtool flex bison gcc-doc gcc-9-multilib gdb-riscv64-linux-gnu gdb-doc gvfs glibc-doc bzip libdv-bin oss-compat
libvisual-0.4-plugins jackd2 opus-tools pcsd libraw1394-doc speex gststreamer1.0-plugins-ugly libstdc++-9-doc samba vde2
debootstrap sharutils-doc bsd-mailx | mailx

```

二、下载实验代码

打开 Ubuntu 终端，一条一条复制执行：

1. 切换目录

```
PowerShell
```

```
$ cd ~
```

2. 下载 xv6 源码仓库

```
PowerShell
```

```
$ git clone git://g.csail.mit.edu/xv6-labs-2021
```

3. 进入代码文件夹

```
PowerShell
```

```
$ cd xv6-labs-2021
```

4. 切换到 util 实验专用分支

```
PowerShell
```

```
$ git checkout util
```

参考：[Lab: Xv6 and Unix utilities](#)（第一个实验）

Fetch the xv6 source for the lab and check out the `util` branch:

```
$ git clone git://g.csail.mit.edu/xv6-labs-2021
Cloning into 'xv6-labs-2021'...
...
$ cd xv6-labs-2021
$ git checkout util
Branch 'util' set up to track remote branch 'util' from 'origin'.
Switched to a new branch 'util'
```

运行示意图：

```
comeat@wdmzhjh: ~/xv6-labs-2021
comeat@wdmzhjh:~/mnt/c/WINDOWS/system32$ cd ~
comeat@wdmzhjh:~$ git clone git://g.csail.mit.edu/xv6-labs-2021
Cloning into 'xv6-labs-2021'...
remote: Enumerating objects: 7051, done.
remote: Counting objects: 100% (7051/7051), done.
remote: Compressing objects: 100% (3423/3423), done.
remote: Total 7051 (delta 3702), reused 6830 (delta 3600), pack-reused 0 (from 0)
Receiving objects: 100% (7051/7051), 17.20 MiB | 1.79 MiB/s, done.
Resolving deltas: 100% (3702/3702), done.
warning: remote HEAD refers to nonexistent ref, unable to checkout.

comeat@wdmzhjh:~$ git clone git://g.csail.mit.edu/xv6-labs-2021
fatal: destination path 'xv6-labs-2021' already exists and is not an empty directory.
comeat@wdmzhjh:~$ cd xv6-labs-2021
comeat@wdmzhjh:~/xv6-labs-2021$ git checkout util
Branch 'util' set up to track remote branch 'util' from 'origin'.
Switched to a new branch 'util'
```

Lab1: Xv6 and Unix utilities

Boot xv6 (easy)

我们可以先构建并运行 xv6：

PowerShell

```
$ make qemu
```

参考：[Lab: Xv6 and Unix utilities（第一个实验）](#)

构建并运行xv6:

```
$ make qemu
riscv64-unknown-elf-gcc -c -o kernel/entry.o kernel/entry.S
riscv64-unknown-elf-gcc -Wall -Werror -O -fno-unit-at-a-time -fcommon -fno-stack-protector -fno-pie -no-pie -c -o kernel/start.o kernel/start.c
...
riscv64-unknown-elf-ld -z max-page-size=4096 -N -e main -Ttext 0 -o user/_zombie user/_zombie.o user/_ulib.o user/_sys.o user/_printf.o user/_umalloc.o
riscv64-unknown-elf-objdump -S user/_zombie > user/_zombie.asm
riscv64-unknown-elf-objdump -t user/_zombie | sed '1,SYMBOL_TABLE/d; s/.* / /; /$/' > user/_zombie.sym
mkfs/mkfs fs.img README user/_xargtest.sh user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertest user/_grind user/_wc user/_zombie
meta 46 0boot, super, log blocks 30 inside blocks 13, bitmap blocks 1 blocks 954 total 1000
balloc: first 591 blocks have been allocated
balloc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,format=raw,id=sd0 -device virtio-blk-device,drive=sd0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

运行示意图:

```
comeat@wdmzhjh: ~/xv6-labs-2021
comeat@wdmzhjh:~/xv6-labs-2021$ make qemu
riscv64-linux-gnu-gcc -c -o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/kalloc.o kernel/kalloc.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/string.o kernel/string.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/main.o kernel/main.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/vm.o kernel/vm.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/proc.o kernel/proc.c
riscv64-linux-gnu-gcc -c -o kernel/switch.o kernel/switch.S
riscv64-linux-gnu-gcc -c -o kernel/trampoline.o kernel/trampoline.S
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/trap.o kernel/trap.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/syscall.o kernel/syscall.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/sysproc.o kernel/sysproc.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/bio.o kernel/bio.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/fs.o kernel/fs.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/log.o kernel/log.c
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/sleeplock.o kernel/sleeplock.c
ser/umalloc.o
riscv64-linux-gnu-objdump -S user/wc > user/wc.asm
riscv64-linux-gnu-objdump -t user/wc | sed '1,/SYMBOL TABLE/d; s/ .* //; /^$/d' > user/wc.sym
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o user/zombie.o user/zombie.c
riscv64-linux-gnu-ld -z max-page-size=4096 -N -e main -Ttext 0 -o user/_zombie user/zombie.o user/ulib.o user/usys.o user/pr
infr.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_zombie > user/zombie.asm
riscv64-linux-gnu-objdump -t user/_zombie | sed '1,/SYMBOL TABLE/d; s/ .* //; /^$/d' > user/zombie.sym
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld -z max-page-size=4096 -N -e main -Ttext 0 -o user/_sleep user/sleep.o user/ulib.o user/usys.o user/prin
tf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_sleep > user/sleep.asm
riscv64-linux-gnu-objdump -t user/_sleep | sed '1,/SYMBOL TABLE/d; s/ .* //; /^$/d' > user/sleep.sym
mkfs/mkfs fs.img README user/xargstest.sh user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln use
r/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_sleep
meta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 954 total 1000
ballocc: first 623 blocks have been allocated
ballocc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,form
at=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
```

输入 ls, 可以看到类似结果:

If you type `ls` at the prompt, you should see output similar to the following:

```
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2059
xargstest.sh 2 3 93
cat        2 4 24256
echo       2 5 23080
forktest   2 6 13272
grep       2 7 27560
init        2 8 23816
kill        2 9 23024
ln          2 10 22880
ls          2 11 26448
mkdir      2 12 23176
rm          2 13 23160
sh          2 14 41976
stressfs    2 15 24016
usertests   2 16 148456
grind       2 17 38144
wc          2 18 25344
zombie      2 19 22408
console     3 20 0
```

 comeat@wdmzhhj: ~/xv6-labs-2021

```
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2226
xargstest.sh 2 3 93
cat        2 4 23896
echo       2 5 22728
forktest   2 6 13088
grep       2 7 27256
init        2 8 23832
kill        2 9 22696
ln          2 10 22656
ls          2 11 26128
mkdir      2 12 22800
rm          2 13 22792
sh          2 14 41664
stressfs    2 15 23800
usertests   2 16 156016
grind       2 17 37968
wc          2 18 25040
zombie      2 19 22192
sleep       2 20 22648
console     3 21 0
```

注：此处有“sleep”是因为在测试之前我已经在相应位置开始尝试写本实验所需要

的文件，可以将“sleep”忽略

sleep (easy)

一、实验目的

实现 sleep 用户工具程序，调用内核 sleep 系统调用使进程暂停指定时钟滴答数，熟悉 xv6 用户程序编写、命令行参数读取、字符串转数字、程序正常退出流程，掌握用户程序编译配置方法。

二、实验步骤

1. 在 user/ 目录新建 sleep.c，实现 main 函数
2. 判断输入参数数量，若 argc != 2，打印错误提示并退出
3. 将 argv[1] 字符串转为整数，传入 sleep() 函数

```
C
sleep.c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
```



```
if(argc != 2){  
    printf("usage: sleep ticks\n");  
    exit(1);  
}  
  
int t = atoi(argv[1]);  
  
sleep(t);  
  
exit(0);  
}
```

4. 修改 Makefile, 将 `sleep` 添加到 UPROGS 编译列表

```
UPROGS=\n    $U/_cat\  
    $U/_echo\  
    $U/_forktest\  
    $U/_grep\  
    $U/_init\  
    $U/_kill\  
    $U/_ln\  
    $U/_ls\  
    $U/_mkdir\  
    $U/_rm\  
    $U/_sh\  
    $U/_stressfs\  
    $U/_usertests\  
    $U/_grind\  
    $U/_wc\  
    $U/_zombie\  
    $U/_sleep\  

```

5. 编译运行: `make qemu`, 输入 `sleep 10` 验证程序休眠 10 秒。

6. 测试结果:

```
comeat@wdmzhjh: ~/xv6-labs-2021
comeat@wdmzhjh: ~/xv6-labs-2021$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sleep 10
$ QEMU: Terminated
comeat@wdmzhjh: ~/xv6-labs-2021$ ./grade-lab-util sleep
make: 'kernel/kernel' is up to date.
== Test sleep, no arguments == sleep, no arguments: OK (1.0s)
== Test sleep, returns == sleep, returns: OK (1.0s)
== Test sleep, makes syscall == sleep, makes syscall: OK (0.7s)
```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验，我对于用户程序调用系统调用的完整流程有了更深入的理解。如果想要实现定时阻塞功能，这可能涉及命令行参数解析、字符串数字转换、sleep 系统调用调用，从而也加深了用户程序与内核交互方式的了解。

pingpong (easy)

一、实验目的

利用 pipe、fork、read、write、getpid 系统调用实现父子进程管道字节收发，理解管道双向通信、进程创建、读写阻塞机制，掌握多进程基础同步逻辑。

二、实验步骤

1. Makefile 添加 \$U/_pingpong\

2. 创建 user/pingpong.c
3. 创建两个管道分别用于父子双向传输
4. 调用 fork 生成子进程，区分父子执行逻辑
5. 子进程读取父管道数据，打印自身 pid 与接收 ping 信息，再写数据传回父进程
6. 父进程等待子发送数据，读取后打印 pong 信息
7. 父子及时关闭不用的文件描述符避免资源占用

```
C++
pingpong.c
#include "kernel/types.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
    int p[2];
    pipe(p);
    char buf;

    int pid = fork();
    if (pid == 0)
    {
        // 子进程：读父发的字节
        read(p[0], &buf, 1);
        printf("%d: received ping\n", getpid());
        // 回写给父进程
        write(p[1], &buf, 1);
        close(p[0]);
        close(p[1]);
    }
```

```

    exit(0);
}
else
{
    // 父进程先发一个字节
    buf = 'x';
    write(p[1], &buf, 1);
    // 等子回复
    read(p[0], &buf, 1);
    printf("%d: received pong\n", getpid());
    close(p[0]);
    close(p[1]);
    wait(0);
    exit(0);
}
}
}

```

8. 测试结果:

```

comeat@wdmzhjh: ~/xv6-labs-2021$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -l. -fno-stack-protector -fno-pie -no-pie -c -o user/pingpong.o user/pingpong.c
riscv64-linux-gnu-ld -z max-page-size=4096 -N -e main -ltext 0 -o user/_pingpong user/pingpong.o user/ulib.o user/usys.o user/_printf.o user/_malloc.o
riscv64-linux-gnu-objdump -S user/_pingpong > user/pingpong.asm
riscv64-linux-gnu-objdump -t user/_pingpong | sed '1,/SYMBOL TABLE/d; s/ .* //; /$/d' > user/pingpong.sym
mkfs/mkfs fs.img README user/xargstest.sh user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_sleep user/_pingpong
mmeta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 954 total 1000
balloc: first 647 blocks have been allocated
balloc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ pingpong
4: received ping
3: received pong
$ QEMU: Terminated
comeat@wdmzhjh: ~/xv6-labs-2021$ ./grade-lab-util pingpong
make: 'kernel/kernel' is up to date.
== Test pingpong == pingpong: OK (1.3s)

```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验，我对于匿名管道与多进程数据交互有了更深的理解。如果想要实现父子进程双向消息传递，这可能涉及管道创建、fork 进程分叉、文件描述符管理、管道阻塞读写，从而也加深了进程间基础同步通信机制的了解。

primes (moderate)/(hard)

一、实验目的

基于 McIlroy 素数筛思想，使用多级管道与并发进程流水线筛选 2~35 全部素数，深入理解管道多级串联、进程并发、文件描述符资源回收、管道 EOF 判断机制。

二、实验步骤

1. Makefile 添加 \$U/_primes\
2. 在 user 目录新建 primes.c，引入 kernel/types.h、user/user.h 头文件，定义递归筛数函数 sieve。
3. 实现 sieve 函数：从传入管道读取整数，读取长度为 0 代表数据流结束，关闭文件描述符退出；读取成功则打印当前素数，新建管道并 fork 子进程。子进程关闭管道写端，递归调用 sieve 处理后续数字；父进程关闭管道读端，过滤掉能被当前素数整除的数值，将剩余数字写入下游管道，读完数据后关闭两端 fd，等待子进程结束再退

出。

3. 编写 main 主函数：创建初始管道，fork 第一个筛分子进程；子进程关闭管道写端，调用 sieve 读取数据流；父进程关闭读端，循环向管道写入 2~35 的整数，写完关闭写端并等待子进程回收。

```
C
primes.c
#include "kernel/types.h"
#include "user/user.h"

void sieve(int readfd)
{
    int p;
    // 读到 0 字节流，结束
    if (read(readfd, &p, sizeof(int)) == 0)
    {
        close(readfd);
        exit(0);
    }
    printf("prime %d\n", p);

    int pipefd[2];
    pipe(pipefd);
    int pid = fork();
    if (pid == 0)
    {
        // 子进程继续筛下一轮
        close(pipefd[1]);
        sieve(pipefd[0]);
    }
}
```

```

    }
    else
    {
        // 父过滤能被 p 整除的数，传给下一级管道
        close(pipefd[0]);
        int n;
        while (read(readfd, &n, sizeof(int)) > 0)
        {
            if (n % p != 0)
            {
                write(pipefd[1], &n, sizeof(int));
            }
        }
        close(readfd);
        close(pipefd[1]);
        wait(0);
        exit(0);
    }
}

int main(void)
{
    int p[2];
    pipe(p);
    int pid = fork();
    if (pid == 0)
    {
        close(p[1]);
        sieve(p[0]);
    }
}

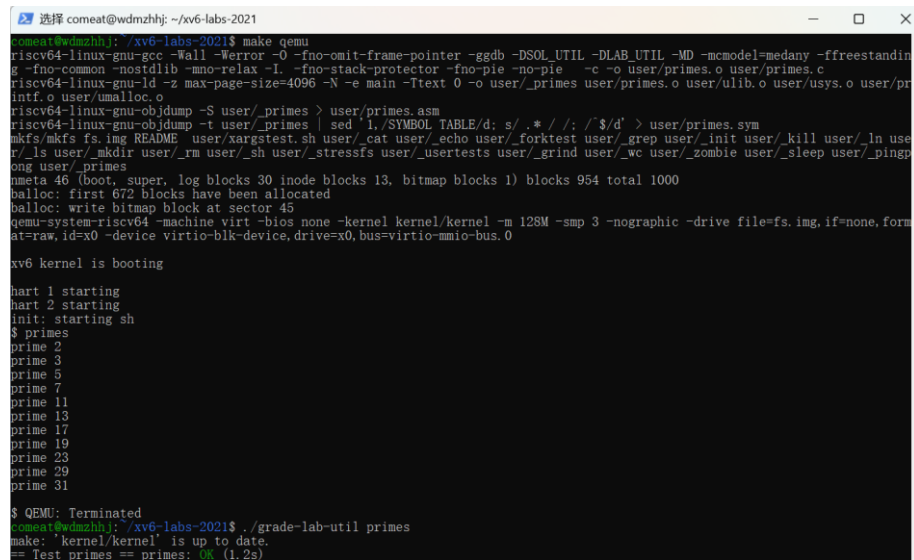
```

```

}
else
{
    close(p[0]);
    // 输入 2~35
    for (int i = 2; i <= 35; i++)
    {
        write(p[1], &i, sizeof(int));
    }
    close(p[1]);
    wait(0);
}
exit(0);
}

```

4. 测试结果:



```

选择 comeat@wdmzhjh: ~/xv6-labs-2021
comeat@wdmzhjh:~/xv6-labs-2021$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -fno-stack-protector -fno-pie -no-pie -c -o user/primos.o user/primos.c
riscv64-linux-gnu-ld -z max-page-size=4096 -N -e main -Ttext 0 -o user/_primos user/primos.o user/ulib.o user/usys.o user/pr
intf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_primos > user/primos.asm
riscv64-linux-gnu-objdump -t user/_primos | sed '1,/SYMBOL TABLE/d; s/.* //; /$/d' > user/primos.sym
mkfs/mkfs fs.img README user/xargstest.sh user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln use
r/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_sleep user/_pingp
ong user/_primos
mmeta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 954 total 1000
balloc: first 672 blocks have been allocated
balloc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,form
at=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
$ QEMU: Terminated
comeat@wdmzhjh:~/xv6-labs-2021$ ./grade-lab-util primes
make: 'kernel/kernel' is up to date.
== Test primes == primes: 31 (1.2s)

```


三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验，我对于多级流水线管道并发模型有了更深的理解。如果想要用多进程管道完成数据过滤，这可能涉及动态创建多级管道、递归 fork 子进程、管道 EOF 识别、大量文件描述符管控，从而也加深并发进程资源管理、流式数据处理机制的了解。

find (moderate)

一、实验目的

实现简化版 find 工具，递归遍历目录树匹配指定文件名，学习目录读取、字符串比较、递归遍历、. 与.. 特殊目录过滤方法。

二、实验步骤

1. Makefile 添加 \$U/_find\
2. 在 user 目录新建 find.c, 引入 types.h、stat.h、user.h、fs.h、fcntl 相关头文件。
3. 定义递归查找函数 find: 打开目标路径, 通过 fstat 获取文件元数据; 若是普通文件, 截取路径末尾文件名与目标字符串比对, 匹配则打印完整路径; 若是目录, 拼接子路径, 过滤. 和 .. 后递归遍历子目录。

4. 编写 main 函数，校验命令行参数数量，参数错误输出使用说明，参数合法则调用查找函数。

```
C
find.c

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void find(char *path, char *target)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    // 打开目录
    if((fd = open(path, O_RDONLY)) < 0){
        fprintf(2, "find: cannot open %s\n", path);
        return;
    }

    // 获取文件状态
    if(fstat(fd, &st) < 0){
        fprintf(2, "find: cannot stat %s\n", path);
        close(fd);
        return;
    }
}
```

```

}

switch(st.type){
case T_FILE:
{
    // 如果是文件，比较文件名
    // 从路径中提取文件名
    char *last_slash = path;
    for(p = path; *p; p++){
        if(*p == '/')
            last_slash = p + 1;
    }
    if(strcmp(last_slash, target) == 0){
        printf("%s\n", path);
    }
    break;
}

case T_DIR:
{
    // 如果是目录，递归遍历
    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
        fprintf(2, "find: path too long\n");
        break;
    }

    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';
}

```

```

while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
        continue;

    // 跳过 "." 和 ".."
    if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
        continue;

    // 构建完整路径
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;

    // 递归查找
    find(buf, target);
}
break;
}
}

close(fd);
}

int main(int argc, char *argv[])
{
    if(argc != 3){
        fprintf(2, "usage: find <directory> <filename>\n");
        exit(1);
    }
}

```

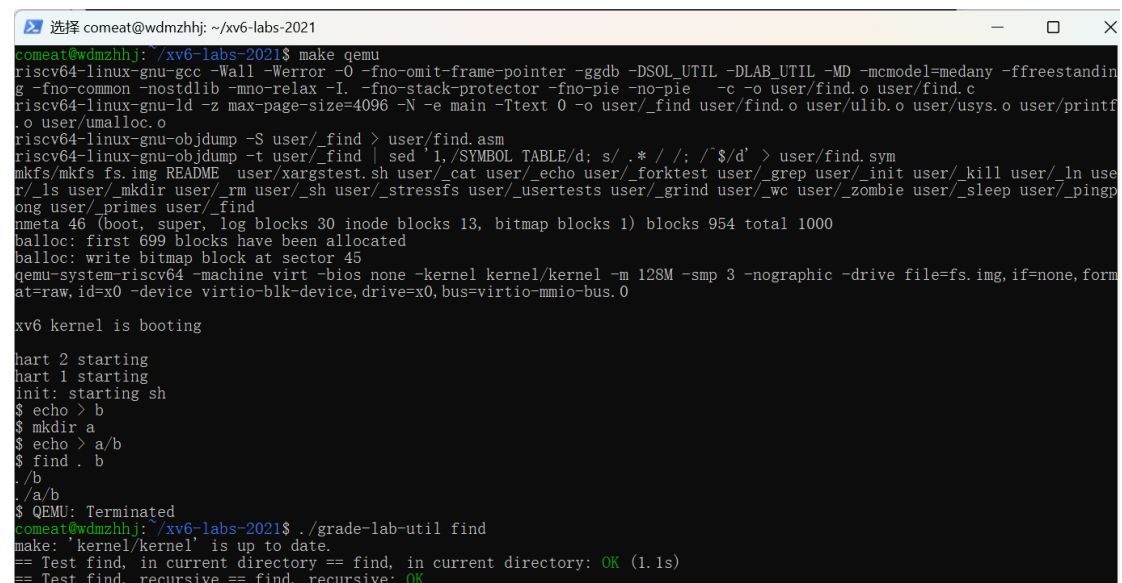
```

find(argv[1], argv[2]);

exit(0);
}

```

5. 测试结果：



```

选择 comeat@wdmzhjh: ~/xv6-labs-2021
comeat@wdmzhjh:~/xv6-labs-2021$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -DLAB_UTIL -MD -mcmmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -L. -fno-stack-protector -fno-pie -no-pie -c -o user/find.o user/find.c
riscv64-linux-gnu-ld -z max-page-size=4096 -N -e main -Ttext 0 -o user/_find user/find.o user/ulib.o user/usys.o user/print.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_find > user/find.asm
riscv64-linux-gnu-objdump -t user/_find | sed '1,/SYMBOL TABLE/d; s/ .* //; /^$/d' > user/find.sym
mkfs/mkfs fs.img README user/xargstest.sh user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_sleep user/_pingpong user/_primes user/_find
nmeta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 954 total 1000
ballocc: first 699 blocks have been allocated
ballocc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b
./b
./a/b
$ QEMU: Terminated
comeat@wdmzhjh:~/xv6-labs-2021$ ./grade-lab-util find
make: 'kernel/kernel' is up to date.
== Test find, in current directory == find, in current directory: OK (1.1s)
== Test find, recursive == find, recursive: OK

```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验，我对于文件系统目录遍历与递归处理有了更深的理解。如果想要递归检索目录下指定文件，这可能涉及目录 inode 读取、字符串对比、递归分支处理、特殊目录过滤，从而也加深 xv6 文件系统基础接口使用的了解。

xargs (moderate)

一、实验目的

实现简易 xargs，逐行读取标准输入并作为参数启动指定命令，掌握逐行 IO 读取、fork+exec 执行子命令、父进程 wait 等待回收、argv 参数数组构造。

二、实验步骤

1. Makefile 添加 \$U/_xargs\

2. 逐字符读取一行输入

按实验提示用 `read(0, &c, 1)` 读到 `\n` 或 EOF，避免一次 `read` 读入多行导致多次命令被错误合并为一次 `exec`。

```
C
int readline(char *buf, int n) {
    int i = 0;
    char c;
    while(i < n - 1){
        if(read(0, &c, 1) < 1) { /* EOF 处理 */ break; }
        if(c == '\n') break;
        buf[i++] = c;
    }
    buf[i] = '\0';
    return i; // 或 EOF 时返回 -1
}
```

3. 将一行按空白切成若干参数

在行缓冲区内用 `\0` 截断，得到可供 `exec` 使用的 `argv` 指针数组；一行内多个词可作为多个附加参数。

```
C
int split(char *buf, char **argv, int maxargs) {
    // 跳过空格/制表符，记录每个 token 起点，遇空白则写入 '\0'
}
```

4. 对每一行 fork/exec，父进程 wait

命令行已有参数（如 `grep hello`）作为固定前缀；再追加本行切出的参数；子进程 `exec`，父进程 `wait(0)`，保证按行串行执行。

```
C
for(i = 1; i < argc; i++)
    xargv[i-1] = argv[i];    // 固定命令前缀
while(readline(buf, sizeof(buf)) >= 0){
    nadd = split(buf, xargv + nbase, MAXARG - nbase - 1);
    if(nadd == 0) continue;  // 空行跳过
    xargv[nbase + nadd] = 0;
    if(fork() == 0){
        exec(xargv[0], xargv);
        exit(1);
    }
    wait(0);
}
```

5. 测试结果：

- 编译后运行 `sh < xargstest.sh`:

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ sh < xargstest.sh
$ $ $ $ $ $ hello
hello
hello
$ $ _
```

- 运行 `make grade` 测试

```
== Test sleep, no arguments ==
$ make qemu-gdb
sleep, no arguments: OK (2.4s)
== Test sleep, returns ==
$ make qemu-gdb
sleep, returns: OK (0.4s)
== Test sleep, makes syscall ==
$ make qemu-gdb
sleep, makes syscall: OK (0.9s)
== Test pingpong ==
$ make qemu-gdb
pingpong: OK (1.1s)
== Test primes ==
$ make qemu-gdb
primes: OK (0.9s)
== Test find, in current directory ==
$ make qemu-gdb
find, in current directory: OK (1.1s)
== Test find, recursive ==
$ make qemu-gdb
find, recursive: OK (1.1s)
== Test xargs ==
$ make qemu-gdb
xargs: OK (1.0s)
== Test time ==
time: OK
Score: 100/100
```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次 xargs 参数分发工具实验，我对于标准输入流式读取、动态创建子进程执行命令有了更深的理解。在调试中我发现数组长度受 MAXARG 限制，单行读取需要做好缓冲区管控，所以批量执行工具必须等待每一个子进程结束再读取下一行输入，保证命令执行顺序与输出完整。

Lab2: system calls

开始之前，切换到 syscall 分支。

```
PowerShell
git fetch
git checkout syscall
make clean
```

System call tracing (moderate)

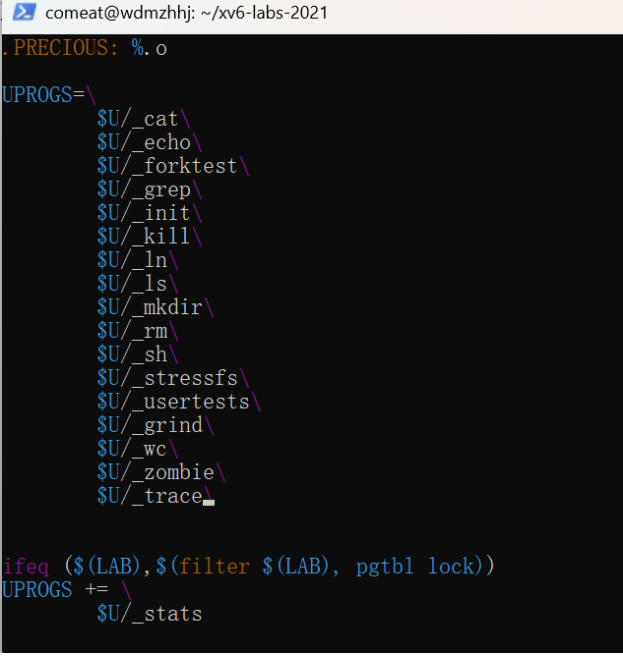
一、实验目的

实现 trace 系统调用，完成系统调用追踪功能。通过新增系统调用、扩展进程控制块、修改系统调用分发逻辑，掌握 xv6 用户态与内核态切换流程、系统调用注册与处

理机制，理解进程 fork 时资源复制规则，实现按掩码筛选打印进程及其子进程的系统调用信息，为后续内核调试提供工具。

二、实验步骤

1. 在 Makefile 中向 UPROGS 添加 \$U/_trace\



```
comeat@wdmzhjh: ~/xv6-labs-2021
.PRECIOUS: %.o

UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_forktest\
    $U/_grep\
    $U/_init\
    $U/_kill\
    $U/_ln\
    $U/_ls\
    $U/_mkdir\
    $U/_rm\
    $U/_sh\
    $U/_stressfs\
    $U/_usertests\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
    $U/_trace\

ifeq ($(LAB),$(filter $(LAB), pgtbl lock))
UPROGS += \
    $U/_stats
```

2. 在 user/user.h 添加声明:

```
C
int trace(int);
```

```

选择 comeat@wdmzhhj: ~/xv6-labs-2021
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int trace(int);

// ulib.c
int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
-- INSERT --

```

3. user/usys.pl 添加系统调用存根:

在文件末尾增加:

```

C
entry("trace");

```

```
选择 comeat@wdmzhj: ~/xv6-labs-2021
print "ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("sleep");
entry("uptime");
entry("trace");
-- INSERT --
```

3. kernel/syscall.h 分配系统调用编号

在文件末尾追加宏定义

C

```
#define SYS_trace 22
```

```
选择 comeat@wdmzhjh: ~/xv6-labs-2021
// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid  11
#define SYS_sbrk    12
#define SYS_sleep   13
#define SYS_uptime  14
#define SYS_open    15
#define SYS_write   16
#define SYS_mknod   17
#define SYS_unlink  18
#define SYS_link    19
#define SYS_mkdir   20
#define SYS_close   21
#define SYS_trace   22
~
-- INSERT --
```

4. 内核实现 sys_trace 处理函数

打开 `kernel/sysproc.c`, 文件末尾添加:

```
C
uint64
sys_trace(void)
{
    int mask;
    // 读取用户传入第一个 int 参数
    argint(0, &mask);
    // 存入当前进程的 trace 掩码
    myproc()->trace_mask = mask;
    return 0;
}
```

```

选择 comeat@wdmzhjh: ~/xv6-labs-2021

// return how many clock tick interrupts have occurred
// since start.
uint64
sys_uptime(void)
{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

uint64
sys_trace(void)
{
    int mask;
    // 读取用户传入的第一个参数
    argint(0, &mask);
    // 存入当前进程的trace掩码
    myproc()->trace_mask = mask;
    return 0;
}

-- INSERT --

```

5. 扩展进程结构体 (kernel/proc.h) 打开 kernel/proc.h, 找到 struct proc {} 结构体内部, 添加成员:

C

```
int trace_mask;
```

```

选择 comeat@wdmzhjh: ~/xv6-labs-2021

// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state; // Process state
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    int xstate;           // Exit status to be returned to parent
    int pid;              // Process ID

    // wait_lock must be held when using this:
    struct proc *parent;  // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;        // Virtual address of kernel stack
    uint64 sz;            // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    struct trapframe *trapframe; // data page for trampoline.S
    struct context context; // swtch() here to run process
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;       // Current directory
    int trace_mask;         // trace系统调用掩码
    char name[16];         // Process name (debugging)
};

-- INSERT --

```

6. 内核注册 sys_trace 函数

打开 kernel/syscall.c

C

文件顶部 extern 声明:

```
extern uint64 sys_trace(void);
```

选择 comeat@wdmzhjhj: ~/xv6-labs-2021

```
extern uint64 sys_exec(void);
extern uint64 sys_exit(void);
extern uint64 sys_fork(void);
extern uint64 sys_fstat(void);
extern uint64 sys_getpid(void);
extern uint64 sys_kill(void);
extern uint64 sys_link(void);
extern uint64 sys_mkdir(void);
extern uint64 sys_mknod(void);
extern uint64 sys_open(void);
extern uint64 sys_pipe(void);
extern uint64 sys_read(void);
extern uint64 sys_sbrk(void);
extern uint64 sys_sleep(void);
extern uint64 sys_unlink(void);
extern uint64 sys_wait(void);
extern uint64 sys_write(void);
extern uint64 sys_uptime(void);
extern uint64 sys_trace(void);

static uint64 (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
-- INSERT --
```

在 syscalls[] 函数指针数组, 对应 SYS_trace 下标添加:

C

```
[SYS_trace] sys_trace,
```

```
选择 comeat@wdmzhjh: ~/xv6-labs-2021
[SYS_uptime] sys_uptime,
[SYS_open] sys_open,
[SYS_write] sys_write,
[SYS_mknod] sys_mknod,
[SYS_unlink] sys_unlink,
[SYS_link] sys_link,
[SYS_mkdir] sys_mkdir,
[SYS_close] sys_close,
[SYS_trace] sys_trace,
};

void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        p->trapframe->a0 = syscalls[num]();
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}
-- INSERT --
```

7. 修改 fork，子进程继承掩码

打开 kernel/proc.c 的 fork() 函数，分配子进程 np 后添加一行复制逻辑：

```
C
np->trace_mask = p->trace_mask;
```

满足需求：父进程 trace 设置会传递给所有后续创建的子进程。


```

选择 comeat@wdmzhjh: ~/xv6-labs-2021
if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
    freeproc(np);
    release(&np->lock);
    return -1;
}
np->sz = p->sz;

// copy saved user registers.
*(np->trapframe) = *(p->trapframe);

// 新增: 复制trace追踪掩码给子进程
np->trace_mask = p->trace_mask;

// Cause fork to return 0 in the child.
np->trapframe->a0 = 0;

// increment reference counts on open file descriptors.
for(i = 0; i < NOFILE; i++)
    if(p->ofile[i])
        np->ofile[i] = filedup(p->ofile[i]);
np->cwd = idup(p->cwd);

safestrcpy(np->name, p->name, sizeof(p->name));

pid = np->pid;
-- INSERT --

```

8. 修改 syscall () 实现打印日志

- 打开 kernel/syscall.c
- 文件全局位置新增系统调用名字数组:

C++

```

static const char *syscall_names[] = {
    [SYS_fork]   "fork",
    [SYS_exit]   "exit",
    [SYS_wait]   "wait",
    [SYS_pipe]   "pipe",
    [SYS_read]   "read",
    [SYS_write]  "write",
    [SYS_open]   "open",
    [SYS_close]  "close",
    [SYS_kill]   "kill",

```

```
[SYS_exec]   "exec",
[SYS_fstat]  "fstat",
[SYS_chdir]  "chdir",
[SYS_dup]    "dup",
[SYS_link]   "link",
[SYS_mkdir]  "mkdir",
[SYS_mknod]  "mknod",
[SYS_unlink] "unlink",
[SYS_getpid] "getpid",
[SYS_sbrk]   "sbrk",
[SYS_sleep]  "sleep",
[SYS_uptime] "uptime",
[SYS_trace]  "trace",
};
```

- 修改 `kernel/syscall.c` 中的 `syscall()` 函数以打印跟踪输出。

```
C
if(p->trace_mask & (1 << num)){
    printf("%d: syscall %s -> %d\n", p->pid, syscall_names[num], p-
>trapframe->a0);
}
```

```

选择 comeat@wdmzhj: ~/xv6-labs-2021
[SYS_unlink] sys_unlink,
[SYS_link] sys_link,
[SYS_mkdir] sys_mkdir,
[SYS_close] sys_close,
[SYS_trace] sys_trace,
};

void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        p->trapframe->a0 = syscalls[num]();
        // 打印跟踪输出
        if(p->trace_mask & (1 << num)){
            printf("%d: syscall %s -> %d\n", p->pid, syscall_names[num], p->trapframe->a0);
        } else {
            printf("%d %s: unknown sys call %d\n",
                p->pid, p->name, num);
            p->trapframe->a0 = -1;
        }
    }
}
-- INSERT --

```

9. 测试结果

- 输入 `make qemu` 后进行测试

PowerShell

trace 32 grep hello README

trace 2147483647 grep hello README

grep hello README

trace 2 usertests forkforkfork

```

$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 968
3: syscall read -> 235
3: syscall read -> 0
$

```

```

$ trace 2147483647 grep hello README
5: syscall trace -> 0
5: syscall exec -> 3
5: syscall open -> 3
5: syscall read -> 1023
5: syscall read -> 968
5: syscall read -> 235
5: syscall read -> 0
5: syscall close -> 0
$

```

```
$ grep hello README
$
```

```
$ trace 2 usertests forkforkfork
usertests starting
9: syscall fork -> 10
test forkforkfork: 9: syscall fork -> 11
11: syscall fork -> 12
12: syscall fork -> 13
12: syscall fork -> 14
13: syscall fork -> 15
12: syscall fork -> 16
14: syscall fork -> 17
```

```
12: syscall fork -> -1
15: syscall fork -> -1
OK
9: syscall fork -> 72
ALL TESTS PASSED
$
```

- 输入 `./grade-lab-syscall trace` 进行测试

```
comeat@wdmzhjhj:~/xv6-labs-2021$ ./grade-lab-syscall trace
make: 'kernel/kernel' is up to date.
== Test trace 32 grep == trace 32 grep: OK (1.3s)
== Test trace all grep == trace all grep: OK (0.9s)
== Test trace nothing == trace nothing: OK (1.0s)
== Test trace children == trace children: OK (8.6s)
comeat@wdmzhjhj:~/xv6-labs-2021$
```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验，我对于用户态到内核态的切换流程、系统调用完整链路有了更深的理解。如果想要实现按掩码过滤打印系统调用日志，这可能涉及用户侧存根函数注册、进程私有标记存储、fork 资源复制、系统调用统一分发打印等操作，从而也加深了进程私有数据生命周期管理的了解。此外这次实验完整走完新增系统调用全流程，我再次巩固 `user.h`、`usys.pl`、`syscall.h`、`syscall.c`、`sysproc.c` 配套修改的标准开发步

骤。除此之外，通过 trace 多场景测试调试，我还明白进程私有追踪掩码必须在 fork 时同步复制给子进程，仅对当前进程及其后代生效，不能全局统一打印；同时需要建立系统调用名称映射数组，在 syscall 分发末尾判断掩码位是否置位，满足条件才输出进程 ID、调用名与返回值，任何注册步骤遗漏都会导致用户程序编译失败、内核识别未知系统调用，完整多进程 fork 追踪测试后才能保证追踪功能隔离稳定。

Sysinfo (moderate)

一、实验目的

实现 sysinfo 系统调用，完成系统运行信息采集。学习内核与用户空间的数据拷贝方法，在内核中统计空闲内存大小与有效进程数量，掌握内核内存管理、进程遍历相关接口，实现将内核统计数据安全传递至用户态，加深对内核资源统计、用户 - 内核数据交互的理解。

二、实验步骤

1. 在 Makefile 中向 UPROGS 添加 \$U/_sysinfotes\

2. 在 user/user.h 添加声明：

```
C
struct sysinfo;
int sysinfo(struct sysinfo *);
```

3. user/usys.pl 在文件末尾增加：

C

```
entry("sysinfo");
```

4. kernel/syscall.h 分配系统调用编号

在文件末尾追加宏定义

C

```
#define SYS_sysinfo 23
```

5. 在 kernel/syscall.c 添加

C

```
extern uint64 sys_sysinfo(void);  
[SYS_sysinfo] sys_sysinfo,  
[SYS_sysinfo] "sysinfo",
```

6. 实现两个辅助函数

- kernel/kalloc.c: 统计空闲内存字节数

文件末尾新增:

C

```
uint64 getfreemem(void) {  
    struct run *r;  
    uint64 cnt = 0;  
    acquire(&kmem.lock);  
    r = kmem.freelist;
```

```

while(r){
    cnt++;
    r = r->next;
}
release(&kmem.lock);
return cnt * PGSIZE; // 页数 × 4KB = 总空闲字节
}

```

- `kernel/proc.c`: 统计非 UNUSED 状态进程数量

```

C
int getnproc(void) {
    int cnt = 0;
    for(int i = 0; i < NPROC; i++){
        if(proc[i].state != UNUSED)
            cnt++;
    }
    return cnt;
}

```

- 在 `kernel/defs.h` 声明两个函数

```

C
uint64 getfreemem(void);
int getnproc(void);

```

7. 实现 `sys_sysinfo()`, 放在 `kernel/sysproc.c`

```

C
#include "sysinfo.h"

uint64
sys_sysinfo(void)
{
    uint64 user_ptr;
    if(argaddr(0, &user_ptr) < 0)
        return -1;

    struct sysinfo info;
    info.freemem = getfreemem();
    info.nproc = getnproc();

    if(copyout(myproc()->pagetable, user_ptr, (char *)&info, sizeof(info)) < 0){
        return -1;
    }
    return 0;
}

```

8. 创建一个新文件, time.txt, 并在里面放一个整数记录完成时间

9. 测试结果

- 我们提供了一个测试程序 sysinfotest ; 如果打印出 " sysinfotest: OK ", 则实验通过。


```
xv6 kernel is booting  
  
hart 2 starting  
hart 1 starting  
init: starting sh  
$ sysinfotest  
sysinfotest: start  
sysinfotest: OK  
$
```

- 运行 make grade

```
== Test trace 32 grep ==  
$ make qemu-gdb  
trace 32 grep: OK (3.0s)  
== Test trace all grep ==  
$ make qemu-gdb  
trace all grep: OK (0.4s)  
== Test trace nothing ==  
$ make qemu-gdb  
trace nothing: OK  
== Test trace children ==  
$ make qemu-gdb  
trace children: OK (7.9s)  
== Test sysinfotest ==  
$ make qemu-gdb  
sysinfotest: OK (1.3s)  
== Test time ==  
time: OK  
Score: 35/35
```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验，我对于内核资源统计、用户与内核空间数据安全拷贝有了更深的理解。如果想要实现向用户程序返回内存、进程全局统计数据，这可能涉及遍历物理内存空闲链表统计总空闲字节、遍历进程列表统计有效进程、使用 copyout 完成内核数据向用户缓冲区拷贝，从而也加深内核内存与进程管理底层接口的了解。此外这次实验复用新增系统调用标准化流程，我再次巩固结构体前置声明、用户指针安全读写、内核内部统计函数封装的实现方式。除此之外，通过 sysinfotest 反复调试，我还明白内核不能直接操作用户传入的结构体指针，必须依靠 copyout 完成内存拷贝防止越界访问；统计空闲内存与活跃进程时要加锁保护全局共享数据，避免多核并发遍历出现数据错乱，同时严格校验用户传入指针合法性，完成基础功能与全量 usertests 综合测试，才能保证系统信息获取功能可靠、不会破坏内核内存安全。

Lab3: page tables

开始之前，切换到 pgtbl 分支

```
PowerShell
git fetch
git checkout pgtbl
make clean
```

Speed up system calls (easy)

一、实验目的

利用共享只读物理页存放进程 PID，避免频繁调用 getpid 陷入内核，减少用户态内核态切换开销，熟悉进程页表映射、物理内存分配与回收流程。

二、实验步骤

1. 在 kernel/proc.h 的 proc 结构体中添加指针来保存这个共享页面的地址

```
C
struct usyscall *usyscallpage;
```

2. kernel/proc.c allocproc 分配物理页并初始化 pid

添加分配代码：

```
C
// 分配一页存放 usyscall 数据
p->usyscallpage = (struct usyscall *)kalloc();
if(p->usyscallpage == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}
// 初始化当前进程 PID 到共享页
p->usyscallpage->pid = p->pid;
```

3. 修改 proc_pagetable(), 建立 USYSCALL 虚拟地址映射

打开 kernel/proc.c, 找到 proc_pagetable 函数, 在 TRAPFRAME 映射完成后, 添加 USYSCALL 页面映射代码, 权限仅可读、允许用户访问 (PTE_R | PTE_U):

```
C
```

```

if(mappages(pagetable, USYSCALL, PGSIZE,
    (uint64)p->usyscallpage, PTE_R | PTE_U) < 0){
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
}

```

4. 修改 freeproc(), 进程销毁释放共享物理页

打开 kernel/proc.c 的 freeproc 函数, 在释放 trapframe 后新增共享页释放逻辑, 释放后置空指针防止野指针:

```

C
if(p->usyscallpage)
    kfree((void*)p->usyscallpage);
p->usyscallpage = 0;

```

5. 测试结果: 运行 ./grade-lab-pgtbl ugetpid 得到如下结果, 实验成功

```

comeat@wdmzhjh: ~/xv6-labs-2021$ ./grade-lab-pgtbl ugetpid
make: 'kernel/kernel' is up to date.
== Test pgtbltest == (0.7s)
== Test   pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK

```

三、实验中遇到的问题和解决办法

1. 页面映射问题: 在实现 ugetpid 共享页面映射时, 遇到页表相关回收问题, 导致内核直接崩溃, 运行测试程序出现如下报错:

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
panic: freewalk: leaf
```

解决方法：通过阅读 xv6 页表管理相关源码，发现除了在 `allocproc()` 和 `freeproc()` 中正确分配、释放共享物理页面之外，进程销毁时用户页表内仍然保留 `USYSCALL` 对应的 PTE 页面映射。进程退出流程会先释放 `usyscallpage` 物理内存，再销毁页表，残留的有效叶子 PTE 指向已回收物理内存，触发 `freewalk` 运行时 panic。因此需要在 `proc_freepagetable` 函数销毁页表前主动取消该虚拟地址映射，添加代码：

```
C
uvmunmap(pagetable, USYSCALL, 1, 0);
```

第四个参数设置为 0，仅清除页表项，不释放物理内存，物理页面交由 `freeproc()` 统一回收，修改后内核不再崩溃，`ugetpid` 测试正常执行。

四、实验心得

本实验让我对虚拟地址、物理内存、进程页表三者的生命周期管理有了完整认知。之前简单认为分配内存时建立映射、释放时回收物理页就足够，实际忽略了页表项与物理内存的绑定关系。当物理页面提前释放，但页表中仍保留有效映射时，页表遍历释放会访问已回收内存，直接触发内核崩溃。同时我理解了共享只读页面的优化思路：把高频读取且不会修改的进程数据放到用户可直接访问的只读物理页，省去频繁 `ecall` 陷入内核的开销，这是操作系统性能优化很经典的手段。资源回收必须遵循“先解除虚拟映射、再释放物理内存”的顺序，虚拟层和物理层资源要成对管理，任何一方遗漏清理都会带来严重内存安全问题。

Print a page table (easy)

一、实验目的

递归遍历三级 RISC-V 页表，打印页表索引、页表项与物理地址，理解 xv6 多级分页存储结构、页表递归遍历逻辑。

二、实验步骤

1. kernel/defs.h 声明 vm 函数原型

在 vm 相关函数区添加：

```
C
void vmprint(pagetable_t;
```

2. kernel/vm.c 递归实现 vmprint

在文件末尾粘贴完整递归代码：

```
C
static void printwalk(pagetable_t pagetable, int depth)
{
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if(pte & PTE_V){
            // 根据层级打印缩进 ..
            if(depth == 2) printf("..");
            else if(depth == 1) printf(".. ..");
        }
    }
}
```

```

else if(depth == 0) printf(".. ..");
uint64 pa = PTE2PA(pte);
printf("%d: pte %p pa %p\n", i, pte, pa);
// 无读写执行位=下一级页表, 递归打印
if((pte & (PTE_R | PTE_W | PTE_X)) == 0){
    printwalk((pagetable_t)pa, depth - 1);
}
}
}
}

void vmprint(pagetable_t pagetable)
{
    printf("page table %p\n", pagetable);
    printwalk(pagetable, 2); // 顶层页表 depth=2
}

```

3. kernel/exec.c 调用打印 init 进程页表

找到 exec 函数最末尾 `return argc;` 上方插入:

```

C
if(p->pid == 1) vmprint(p->pagetable);

```

4. 测试结果

- 编译并运行, 结果输出:

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000
.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000
.. .. ..509: pte 0x0000000021fdd813 pa 0x0000000087f76000
.. .. ..510: pte 0x0000000021fddc07 pa 0x0000000087f77000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
$
```

- 运行 `./grade-lab-pgtbl print`, 得到如下结果, 实验成功

```
comeat@wdmzhjhj: ~/xv6-labs-2021$ ./grade-lab-pgtbl print
make: 'kernel/kernel' is up to date.
== Test pte printout == pte printout: OK (0.8s)
```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过递归打印页表, 我理清了 RISC-V 三级分页的分层结构, 明白了操作系统如何通过三级页表完成虚拟地址到物理地址的转换。递归遍历的设计十分巧妙, 利用权限位区分中间页表和存储数据的叶子页, 不用额外记录层级信息。格式化缩进输出也让我直观看到不同层级页表的包含关系, 不再是单纯纸上理解地址转换公式。同时意识到内核中打印调试是定位内存、页表问题的重要工具, 后续遇到缺页、内存崩溃时, 可以借助自定义页表打印函数快速排查映射异常。

Detecting which pages have been accessed (hard)

一、实验目的

基于硬件 PTE_A 访问标记位实现 `pgaccess` 系统调用，检测一段时间内用户进程访问过的页面，掌握页表项读取、内核向用户空间拷贝数据、系统调用参数解析方法。

二、实验步骤

1. kernel/riscv.h 定义硬件访问位 PTE_A

打开 `riscv.h` PTE 宏定义区，新增：

```
C
#define PTE_A (1L << 6) // RISC-V 页面访问标记位
```

2. kernel/sysproc.c 实现 `sys_pgaccess` 系统调用

文件末尾添加：

```
C
uint64 sys_pgaccess(void)
{
    uint64 va; // 起始虚拟页地址
    int npages; // 需要检测的页面数量
    uint64 buf; // 用户态存放掩码的缓冲区
```

```

// 取出三个系统调用参数
if(argaddr(0, &va) < 0) return -1;
if(argint(1, &npages) < 0) return -1;
if(argaddr(2, &buf) < 0) return -1;

// 限制最多 64 页，只用 1 个 uint64 保存位掩码
if(npages > 64) return -1;
uint64 mask = 0;
struct proc *p = myproc();

for(int i = 0; i < npages; i++){
    uint64 cur_va = va + i * PGSIZE;
    // 查找当前虚拟地址对应的 PTE，不分配页表
    pte_t *pte = walk(p->pagetable, cur_va, 0);
    if(pte == 0) continue;
    // 判断是否设置访问位 PTE_A
    if(*pte & PTE_A){
        mask |= (1UL << i);
        *pte &= ~PTE_A; // 清除访问位，方便下一次检测
    }
}

// 将内核掩码拷贝到用户缓冲区
if(copyout(p->pagetable, buf, (char*)&mask, sizeof(mask)) < 0)
    return -1;
return 0;
}

```

3. 在 kernel/defs.h 中添加 walk 声明

C

```
pte_t* walk(pagetable_t, uint64, int); // 在//vm.c 下添加
```

4. 测试结果

- 编译并运行 `pgtbltest`, 结果输出:

```
$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
pgaccess_test starting
pgaccess_test: OK
pgtbltest: all tests succeeded
$ _
```

- 运行 `./grade-lab-pgtbl pgaccess`, 得到如下结果, 实验成功

```
comeat@wdmzhj:~/xv6-labs-2021$ ./grade-lab-pgtbl pgaccess
make: 'kernel/kernel' is up to date.
== Test pgtbltest == (1.2s)
== Test pgtbltest: pgaccess ==
pgtbltest: pgaccess: OK
```

三、实验中遇到的问题和解决办法

无

四、实验心得

这个实验让我接触到 CPU 硬件提供的页表辅助标记位, 意识到操作系统很多内存管理功能都依托硬件特性实现, 而非纯软件模拟。硬件自动置位 PTE_A 记录页面访问行为, 内核只需读取、清除标记就能统计页面冷热, 为页面置换、内存回收提供依据。

编写系统调用过程中，巩固了用户态参数解析、内核与用户空间安全拷贝的知识点，牢记不能直接在内核解引用用户指针，必须使用 `copyout` 完成数据传输。同时边界处理也十分关键，不对传入页数做上限限制会造成掩码溢出、内存越界，内核开发需要时刻考虑用户传入参数的合法性。

Lab4: traps

开始之前，切换到 `traps` 分支。

```
PowerShell
git fetch
git checkout traps
make clean
```

RISC-V assembly (easy)

xv6 仓库里有一个 `user/call.c` 文件，运行 `make fs.img` 能对其进行编译，并生成 `user/call.asm` 中程序的可读汇编版本。

Q1:

Which registers contain arguments to functions? For example, which register holds 13 in main's call to `printf`?

A:

(1) 函数的参数存放在 `a0`、`a1`、`a2` 等寄存器中

(2) 13 放在寄存器 a2 中:

```
void main(void) {
  1c: 1141          addi sp,sp,-16
  1e: e406          sd   ra,8(sp)
  20: e022          sd   s0,0(sp)
  22: 0800          addi s0,sp,16
  printf("%d %d\n", f(8)+1, 13);
  24: 4635          li    a2,13
  26: 45b1          li    a1,12
  28: 00000517      auipc  a0,0x0
  2c: 7b050513      addi a0,a0,1968 # 7d8 <malloc+0xea>
  30: 00000097      auipc  ra,0x0
  34: 600080e7      jalr  1536(ra) # 630 <printf>
  exit(0);
  38: 4501          li    a0,0
  3a: 00000097      auipc  ra,0x0
  3e: 27e080e7      jalr  638(ra) # 2b8 <exit>
}
```

Q2:

Where is the call to function `f` in the assembly code for `main`? Where is the call to `g`? (Hint: the compiler may inline functions.)

A:

程序中并未出现调用 `f`、`g` 的跳转指令，编译器对两个函数均做了常量内联优化：传入参数 8 时， $g(x)=x+3$ 、 $f(x)=g(x)$ 的计算逻辑可在编译期直接算出结果 12，无需运行时跳转执行函数，因此汇编代码直接通过 `li a1, 12` 将常量 12 载入寄存器，省去了对 `f`、`g` 的函数调用流程。

```

void main(void) {
1c: 1141          addi sp,sp,-16
1e: e406          sd   ra,8(sp)
20: e022          sd   s0,0(sp)
22: 0800          addi s0,sp,16
printf("%d %d\n", f(8)+1, 13);
24: 4635          li    a2,13
26: 45b1          li    a1,12
28: 00000517      auipc  a0,0x0
2c: 7b050513      addi a0,a0,1968 # 7d8 <malloc+0xea>
30: 00000097      auipc  ra,0x0
34: 600080e7      jalr  1536(ra) # 630 <printf>
exit(0);
38: 4501          li    a0,0
3a: 00000097      auipc  ra,0x0
3e: 27e080e7      jalr  638(ra) # 2b8 <exit>

```

Q3:

At what address is the function `printf` located?

A:

地址在: 0x630

```

void
printf(const char *fmt, ...)
{
630:711d          addi sp,sp,-96
632:ec06          sd   ra,24(sp)
634:e822          sd   s0,16(sp)
636:1000          addi s0,sp,32

```

```

void main(void) {
1c: 1141          addi sp,sp,-16
1e: e406          sd   ra,8(sp)
20: e022          sd   s0,0(sp)
22: 0800          addi s0,sp,16
printf("%d %d\n", f(8)+1, 13);
24: 4635          li    a2,13
26: 45b1          li    a1,12
28: 00000517      auipc   a0,0x0
2c: 7b050513      addi a0,a0,1968 # 7d8 <malloc+0xea>
30: 00000097      auipc   ra,0x0
34: 600080e7      jalr   1536(ra) # 630 <printf>
exit(0);
38: 4501          li    a0,0
3a: 00000097      auipc   ra,0x0
3e: 27e080e7      jalr   638(ra) # 2b8 <exit>

```

Q4:

What value is in the register `ra` just after the `jalr` to `printf` in `main`?

A:

- `PC=0x30` 时, 由 $ra = pc + (0x0 \ll 12)$ 计算出 $ra=0x30+0x0=0x30$
 - `PC=34` 时, `jalr 1536(ra)` 跳转到偏移地址 `printf` 处, 也就是 $0x30+1536=0x630$ 的位置, 然后再将返回地址 $PC + 4 = 0x34 + 4 = 0x38$ 存入 `ra`
- 所以 `jalr` 指令执行完后, `ra` 的值为 `0x38`

```

void main(void) {
  1c: 1141          addi sp,sp,-16
  1e: e406          sd   ra,8(sp)
  20: e022          sd   s0,0(sp)
  22: 0800          addi s0,sp,16
  printf("%d %d\n", f(8)+1, 13);
  24: 4635          li    a2,13
  26: 45b1          li    a1,12
  28: 00000517      auipc   a0,0x0
  2c: 7b050513      addi a0,a0,1968 # 7d8 <malloc+0xea>
  30: 00000097      auipc   ra,0x0
  34: 600080e7      jalr  1536(ra) # 630 <printf>
  exit(0);
  38: 4501          li    a0,0
  3a: 00000097      auipc   ra,0x0
  3e: 27e080e7      jalr  638(ra) # 2b8 <exit>

```

Q5:

Run the following code.

```

C
unsigned int i = 0x00646c72;
printf("H%x Wo%s", 57616, &i);

```

What is the output? [Here's an ASCII table](#) that maps bytes to characters.

The output depends on that fact that the RISC-V is little-endian. If the RISC-V were instead big-endian what would you set to in order to yield the same output? Would you need to change to a different value? `57616`

[Here's a description of little- and big-endian](#) and [a more whimsical description](#).

A:

步骤:

1. 在 `user/` 新建文件 `lab4_test.c`

```
C++  
  
#include "kernel/types.h"  
#include "user/user.h"  
  
int main(void)  
{  
    unsigned int i = 0x00646c72;  
    printf("H%x Wo%s", 57616, &i);  
    exit(0);  
}
```

2. 修改 `Makefile` 找到 `UPROGS`, 在里面添加 `$U/_lab4_test` 示例:
3. 编译启动 `make qemu`
4. 在 `xv6 shell` 输入命令运行 `lab4_test`

输出结果: `HE110 World`

```
xv6 kernel is booting  
  
hart 2 starting  
hart 1 starting  
init: starting sh  
$ lab4_test  
HE110 World$ _
```

- 疑惑的点: 按理来说, `%x` 格式符输出的应该是小写的 16 进制, 但是这里 `"HE110"` 中的 `E` 却是大写的

- 原因：查看 `user/printf.c` 内容发现，在数组里字母是大写 ABCDEF，在打印时没有区分 X 与 x，所以在输出十六进制的字母时均为大写

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

#include <stdarg.h>

static char digits[] = "0123456789ABCDEF";

static void
putc(int fd, char c)
{
    write(fd, &c, 1);
}
```

```
c = int[i] & 0xFF;
if(state == 0){
    if(c == '%'){
        state = '%';
    } else {
        putc(fd, c);
    }
} else if(state == '%'){
    if(c == 'd'){
        printint(fd, va_arg(ap, int), 10, 1);
    } else if(c == 'l') {
        printint(fd, va_arg(ap, uint64), 10, 0);
    } else if(c == 'x') {
        printint(fd, va_arg(ap, int), 16, 0);
    } else if(c == 'p') {
        printptr(fd, va_arg(ap, uint64));
    } else if(c == 's'){
        s = va_arg(ap, char*);
        if(s == 0)
            s = "(null)";
        while(*s != 0){
            putc(fd, *s);
            s++;
        }
    }
}
```

分析：

(1) 首先，根据 `user/printf.c` 中的数组可知，57616 转为 16 进制后输出为 E110

(2) 0x00646c72 转为 ASCII 对应如下

Hx	00	64	6c	72
----	----	----	----	----

ASCII	/0	d	l	r
-------	----	---	---	---

- 小端序：最低有效字节，放在最低内存地址。

内存地址由低 → 高存放顺序：0x72 → 0x6c → 0x64 → 0x00

即输出 rld

- 大端序：最高有效字节，放在最低内存地址

如果直接使用 `i = 0x00646c72`，内存排布：0x00 → 0x64 → 0x6c → 0x72

第一个读到 0x00（字符串结束符），直接终止，只能打印 Wo，无法得到 World、

所以，如果 RISC-V 是大端序，应该将 `i` 设置为 0x726c6400，不需要修改 57616，因为其无论在大端序还是小端序，都是作为 16 进制数打印的

验证：在小端序中模拟大端序的输出

```
C++
user/lab4_test2.c
#include "kernel/types.h"
#include "user/user.h"

int main(void)
{
    unsigned int i = 0x726c6400;
    unsigned char *p = (unsigned char *)&i;
    printf("byte0=%x byte1=%x byte2=%x byte3=%x\n", p[0], p[1], p[2], p[3]);
}
```

```
printf("H%x Wo%s", 57616, &i);  
exit(0);  
}
```

输出:

```
xv6 kernel is booting  
  
hart 1 starting  
hart 2 starting  
init: starting sh  
$ lab4_test2  
byte0=0 byte1=64 byte2=6C byte3=72  
HE110 Wo$ _
```

由结果可知，如果在大端序中仍用原来的 `i` 值，那么只能输出 `Wo`，与分析推论一致

Q6:

In the following code, what is going to be printed after ? (note: the answer is not a specific value.) Why does this happen? 'y='

```
C  
printf("x=%d y=%d", 3);
```

A:

在 `y=` 之后会打印一个不确定的随机值，这是因为 `printf` 的格式字符串中包含两个 `%d` 格式说明符，却只传入了一个参数 (3)，导致第二个 `%d` 没有对应的参数可供

匹配。printf 是可变参数函数，它会根据格式字符串中的占位符数量从栈或寄存器中取参数，因此缺少参数时会读取到内存或寄存器中的旧数据或垃圾值，从而产生未定义行为，打印出的值不可预测，每次运行都可能不同。

Backtrace (moderate)

一、实验目的

基于栈帧指针 s0 实现内核栈回溯函数，利用栈帧布局逐层打印函数返回地址，掌握内核栈结构、内联汇编读取寄存器、栈边界判断方法，用于内核崩溃调试。

二、实验步骤

1. 打开 kernel/riscv.h 文件，在任意宏/函数区域添加这段汇编函数：

```
C
static inline uint64
r_fp()
{
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x) );
    return x;
}
```

- 作用：读取 RISC-V s0 寄存器（编译器保存的当前栈帧指针 fp），给 backtrace 使用。

2. 打开 kernel/defs.h，找到内核函数声明区，添加函数原型：

C

```
void backtrace(void);
```

- 作用：让 `sysproc.c`、`printf.c` 的 panic 函数能调用 `backtrace`，消除隐式声明报错。

3. 打开 `kernel/printf.c`，在文件末尾新增完整 `backtrace` 代码：

C

```
void backtrace(void)
{
    uint64 fp = r_fp();
    printf("backtrace:\n");
    // 循环条件：fp 必须在当前内核栈单页内，超出栈页停止遍历
    while(PGROUNDDOWN(fp) == PGROUNDDOWN(fp)){
        // fp-8：栈帧中保存的函数返回地址 ra
        uint64 ra = *(uint64 *)(fp - 8);
        printf("%p\n", ra);
        // fp-16：上一级调用者的栈帧指针
        fp = *(uint64 *)(fp - 16);
        // 退出循环：帧指针走到栈页面底部，停止，防止越界死循环
        if(fp == PGROUNDDOWN(fp))
            break;
    }
}
```

核心原理：

- RISC-V 栈帧规则：`fp-8` 存返回地址、`fp-16` 存父帧指针；
- `xv6` 每个内核栈独占 1 页(4KB)，用 `PGROUNDDOWN` 限制遍历范围，避免访问

非法内存。

4. 修改 `kernel/sysproc.c` 的 `sys_sleep`, 插入 `backtrace` 调用

- 打开 `kernel/sysproc.c`, 找到 `sys_sleep` 函数;
- 在函数最开头添加 `backtrace();`:

```
C
uint64
sys_sleep(void)
{
    backtrace(); // 新增栈回溯打印
    int n;
    uint ticks0;
    if(argint(0, &n) < 0)
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        if(myproc()->killed){
            release(&tickslock);
            return -1;
        }
        sleep(&ticks, &tickslock);
    }
    release(&tickslock);
    return 0;
}
```

测试程序 `bttest` 会调用 `sleep` 系统调用, 触发 `sys_sleep`, 自动打印调用栈。

5. 测试结果

- 编译运行后输入 `bttest` 进行测试

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ bttest
backtrace:
0x00000000800020cc
0x0000000080001fa6
0x0000000080001c90
$
```

- 运行 `bttest` 之后退出 `qemu`, 运行 `addr2line -e kernel/kernel`, 并剪切粘贴上述地址, 则会得到如下所示的结果:

```
comeat@wdmzhjhj:~/xv6-labs-2021$ addr2line -e kernel/kernel
0x00000000800020cc
/home/comeat/xv6-labs-2021/kernel/sysproc.c:62
0x0000000080001fa6
/home/comeat/xv6-labs-2021/kernel/syscall.c:140
0x0000000080001c90
/home/comeat/xv6-labs-2021/kernel/trap.c:76
```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验, 我对于内核栈帧结构与栈回溯原理有了更深的理解。如果需要实现栈回溯功能, 这可能涉及读取 CPU 帧指针寄存器、解析固定偏移下的返回地址与上层栈帧指针, 从而也加深了对函数调用栈存储布局的了解。此外这次实验也涉及到用

户态与内核态的切换、内核内存地址边界判断，我再次巩固了内联汇编读取寄存器、页边界宏校验等内核开发操作。除此之外，通过这一次的调试测试，我还明白，在修改内核代码新增调试功能的时候，应当确保不破坏原有系统稳定性，即在遍历栈时做好地址范围限制，防止无限循环、越界访问内存，同时保证新增函数可以正常嵌入 panic 流程，帮助快速定位内核崩溃位置。进行这样充分的边界测试，我们才能确保栈回溯功能不会引发内存访问异常、系统卡死等问题。

Alarm (hard)

一、实验目的

新增 igalarm、sigreturn 系统调用，实现进程定时用户态告警回调，掌握时钟中断处理、陷阱上下文保存恢复、用户态与内核态切换、进程状态扩展、防止重入告警的机制。

二、实验步骤

1. Makefile 添加测试程序

打开根目录 Makefile，在 UPROGS 列表追加：

```
Plain Text
$U/_alarmtest\
```

2. user/user.h 添加函数声明

在系统调用区域添加两行：

C

```
int sigalarm(int ticks, void (*handler)());  
int sigreturn(void);
```

3. user/usys.pl (生成汇编 sys 调用)

添加两行 entry, 脚本会自动生成 usys.S:

Perl

```
entry("sigalarm");  
entry("sigreturn");
```

4. kernel/syscall.h 新增系统调用号

在末尾追加:

C

```
#define SYS_sigalarm 22  
#define SYS_sigreturn 23
```

5. kernel/syscall.c 外部函数声明:

C

```
extern uint64 sys_sigalarm(void);  
extern uint64 sys_sigreturn(void);
```

6. syscalls 数组填入映射:

C

```
[SYS_sigalarm] sys_sigalarm,  
[SYS_sigreturn] sys_sigreturn,
```

7. 修改 proc.h, 给进程结构体新增 alarm 字段

打开 kernel/proc.h 的 struct proc, 插入 4 个成员:

C

```
int alarm_interval;  
uint64 handler_va;  
int passed_ticks;  
int have_return;  
struct trapframe saved_trapframe;
```

字段作用:

- `alarm_interval`: sigalarm 系统调用传入的触发周期 tick 数, 值为 0 时关闭定时警报功能;
- `handler_va`: 用户传入 alarm 处理函数的虚拟内存地址, 定时器触发后跳转至此地址执行;
- `passed_ticks`: 记录当前进程持续占用 CPU 的时钟 tick 累计值, 每一次硬件时钟中断自增 1;
- `have_return`: 标记警报处理函数状态, 1 代表上一次 handler 已经执行完毕、允许下次触发; 0 代表 handler 正在运行, 禁止嵌套触发 (对应 test2 防重入需求);
- `saved_trapframe`: 完整存储触发定时器中断瞬间的用户寄存器现场 (trapframe 全部成员), sigreturn 系统调用会读取该结构体恢复被打断的程序执行上下文。

8. proc.c 初始化新增字段

打开 kernel/proc.c 的 allocproc()函数 (新建进程时初始化), 在变量初始化处添加:

```
C
p->alarm_interval = 0;
p->handler_va = 0;
p->passed_ticks = 0;
p->have_return = 1;
```

9. 实现两个系统调用 (kernel/sysproc.c)

- sys_sigalarm 实现

```
C
uint64
sys_sigalarm(void)
{
    struct proc *p = myproc();
    int n;
    uint64 handler;
    // 取第一个参数: 间隔 tick
    if(argint(0, &n) < 0)
        return -1;
    // 取第二个参数: 函数指针地址
    if(argaddr(1, &handler) < 0)
        return -1;

    p->alarm_interval = n;
    p->handler_va = handler;
    p->passed_ticks = 0;
    p->have_return = 1;
```

```
return 0;
}
```

- sys_sigreturn 实现 (恢复用户现场)

```
C
uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    // 把保存的中断现场完整还原到当前 trapframe
    *p->trapframe = p->savd_trapframe;
    p->have_return = 1;
    return 0;
}
```

10. 修改 trap.c 的 usertrap() 时钟中断逻辑 (核心触发逻辑)

usertrap 是用户态陷阱处理函数, 硬件时钟中断号 `which_dev == 2`, 在这里统计 tick 并触发 alarm:

找到 usertrap 里的时钟中断分支, 修改代码:

```
C
if(which_dev == 2){
    struct proc *p = myproc();
    if(p->alarm_interval != 0 && p->have_return){
        p->passed_ticks++;
        if(p->passed_ticks == p->alarm_interval){
            p->savd_trapframe = *p->trapframe;
            p->trapframe->epc = p->handler_va;
        }
    }
}
```

```

    p->passed_ticks = 0;
    p->have_return = 0;
}
}
yield();
}

```

逻辑说明：

- (1) `which_dev == 2`：硬件定时器产生时钟中断的固定识别标识，进入 tick 处理分支；
- (2) 加锁修改全局 `ticks` 变量并唤醒等待 `sleep` 的进程，保证全局计时同步；
- (3) `p->alarm_interval != 0`：判断进程是否启用定时警报，0 代表关闭，直接跳过逻辑；
- (4) `p->have_return`：判断当前无正在执行的警报处理函数，避免嵌套触发；
- (5) 两个条件同时满足时，进程 CPU 计时 `passed_ticks` 自增；
- (6) 累计 tick 等于设定周期时：
 - 拷贝当前全部用户寄存器现场保存至 `saved_trapframe`；
 - 修改陷阱帧 `epc`（程序计数器），内核返回用户态时直接跳转到 `handler` 函数；
 - 清空计时计数器，实现周期性重复触发；
 - 将 `have_return` 置 0，锁定状态，中断期间不会再次触发警报；
- (7) 用户 handler 执行完调用 `sigreturn`，会把 `have_return` 重新置 1，解除锁定，等待下一轮计时触发

11. 修改 `kernel/param.h`，将磁盘容量扩大，测试完成后再修改回原值

C

```
#define FSSIZE    20000
```

12. 测试结果:

- 编译运行后输入 `alarmtest` 进行测试

```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ alarmtest
test0 start
.....alarm!
test0 passed
test1 start
....alarm!
...alarm!
...alarm!
...alarm!
...alarm!
....alarm!
...alarm!
....alarm!
....alarm!
....alarm!
.test1 passed
test2 start
.....alarm!
test2 passed
```

- 运行 `usertests` 进行测试

```
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$ _
```

显示上述内容，说明测试成功

三、实验中遇到的问题和解决办法

1. 运行 `usertests` 执行到 `writebig` 时触发内核崩溃，报错 `panic: balloc: out of blocks`，提示磁盘块分配耗尽。

```
test opentest: OK
test writetest: OK
test writebig: panic: balloc: out of blocks
```

- 原因：xv6 文件系统默认磁盘总块数定义在 `kernel/param.h` 中：`#define FSSIZE 1000`。磁盘容量较小，`writebig` 测试会持续写入大量文件数据，很快占满全部可用磁盘块，触发块分配失败。
- 解决办法：修改 `kernel/param.h`，将磁盘容量扩大：

```
C
#define FSSIZE    20000
```

退出 QEMU 虚拟机，执行完整清理重建命令，必须先 `make clean` 删除旧磁盘镜像，否则扩容不生效：

```
PowerShell
make clean
make
make qemu
```

此时重新运行 `usertests` 即可

四、实验心得

通过本次实验，我对于定时中断的处理有了更深的理解。如果需要实现定时中断处理并触发用户自定义回调函数，这可能涉及到操作硬件时钟滴答计数、陷阱上下文保存与恢复，从而也加深了对中断处理机制、用户态陷阱流程的了解。此外这次实验也涉及到用户和内核两态的转换，我再次巩固了如何新增、声明、注册系统调用，搭

建用户程序与内核处理逻辑之间的连接。除此之外，通过这一次多组测试用例调试，我还明白，在修改内核拓展进程功能的时候，应当确保不影响系统稳定性，即在实现定时告警功能时，要完整保存并恢复用户全部寄存器上下文、增加告警重入保护标记，确保中断处理程序能够及时返回，避免嵌套中断、流程错乱，干扰其他进程调度与系统正常中断响应。进行这样充分的多场景测试，我们才能确保定时告警功能不会导致程序卡死、内核崩溃或执行流程异常。

Lab5: Copy-on-Write Fork for xv6

开始之前，切换到 `cow` 分支。

```
PowerShell
git fetch
git checkout cow
make clean
```

Implement copy-on write(hard)

一、实验目的

本实验基于 xv6 操作系统实现写时复制（COW）机制，对比原版 fork 完整拷贝内存的性能缺陷，依托 RISC-V 页表软件位标记共享页面，搭建物理页引用计数管理体系，同步修改页表拷贝、缺页陷阱、内核内存分配、用户数据拷贝等内核模块，实现 fork 仅复制页表、写入时延迟分配私有内存的惰性优化，通过专项与全量测试验证功能完整性，掌握共享资源生命周期管理、硬件异常捕获、内核多子系统协同开发的底层原理。

二、实验步骤

1. 定义 COW 专用页表标记位

复用 RISC-V 预留 RSW 软件位区分 COW 共享页与普通只读页，修改 `kernel/riscv.h`

```
C
#define PTE_COW (1L << 8) // 写时复制标记，软件可用位
```

2. 同一物理页可被多进程共享，通过数组记录引用数量，只有计数归零时释放页面，修改 `kernel/kalloc.c`、`kernel/defs.h`

- `kmem` 结构体新增计数数组

```
C
struct {
    struct spinlock lock;
    struct run *freelist;
    int refcnt[PHYSTOP / PGSIZE]; // 物理页引用计数
} kmem;
```

- `kalloc` 分配页面时计数置 1

```
C
if(r){
    memset((char*)r, 5, PGSIZE);
    uint64 idx = (uint64)r / PGSIZE;
    kmem.refcnt[idx] = 1;
}
```

- `kfree` 释放时仅计数归零才回收内存

C

```
uint64 idx = (uint64)pa / PGSIZE;
kmem.refcnt[idx]--;
if(kmem.refcnt[idx] > 0){
    release(&kmem.lock);
    return;
}
// 计数为 0，归还空闲链表
```

- 新增计数增减、查询函数，并在 defs.h 声明

C

```
// kalloc.c
void krefinc(void *pa){
    acquire(&kmem.lock);
    uint64 idx = (uint64)pa / PGSIZE;
    kmem.refcnt[idx]++;
    release(&kmem.lock);
}

int krefcnt(void *pa){
    acquire(&kmem.lock);
    uint64 idx = (uint64)pa / PGSIZE;
    int cnt = kmem.refcnt[idx];
    release(&kmem.lock);
    return cnt;
}

// defs.h 添加声明
void krefinc(void *);
int krefcnt(void *);
```

```
int cowfault(pagetable_t, uint64);
```

3. 删除原版全量内存拷贝逻辑，父子共用物理页，清除写权限、打上 COW 标记，`kernel/vm.c`

```
C
for(i = 0; i < sz; i += PGSIZE){
    pte = walk(old, i, 0);
    uint64 pa = PTE2PA(*pte);
    uint flags = PTE_FLAGS(*pte);
    // 可写页面开启 COW
    if(flags & PTE_W){
        flags = (flags & ~PTE_W) | PTE_COW;
        *pte = PA2PTE(pa) | flags;
    }
    krefinc((void*)pa); // 共享计数+1
    // 子进程映射同一物理地址，不分配新内存
    if(mappages(new, i, PGSIZE, pa, flags) != 0)
        goto err;
}
```

4. 新增 `cowfault`，处理写缺页私有页分配：写入 COW 页触发缺页时，计数为 1 直接恢复写权限；计数大于 1 分配新页拷贝数据

```
C
int cowfault(pagetable_t pagetable, uint64 va){
    va = PGROUNDDOWN(va);
    pte_t *pte = walk(pagetable, va, 0);
```

```

if(!(*pte & PTE_COW)) return -1;
uint64 pa = PTE2PA(*pte);
uint flags = PTE_FLAGS(*pte);
// 页面仅自身引用，无需拷贝
if(krefcnt((void*)pa) == 1){
    *pte = PA2PTE(pa) | ((flags | PTE_W) & ~PTE_COW);
    return 0;
}
// 多进程共享，分配新页复制内容
char *mem = kalloc();
if(mem == 0) return -1;
memmove(mem, (char*)pa, PGSIZE);
*pte = PA2PTE((uint64)mem) | ((flags | PTE_W) & ~PTE_COW);
kfree((void*)pa); // 旧页计数减 1
return 0;
}

```

5. 修改 usertrap 捕获存储缺页异常：RISC-V 异常码 15 代表存储写故障，调用 cowfault 处理 COW 页面， kernel/trap.c

```

C
else if(r_scause() == 15){
    uint64 va = r_stval();
    if(va >= p->sz || cowfault(p->pagetable, va) < 0)
        p->killed = 1;
}

```

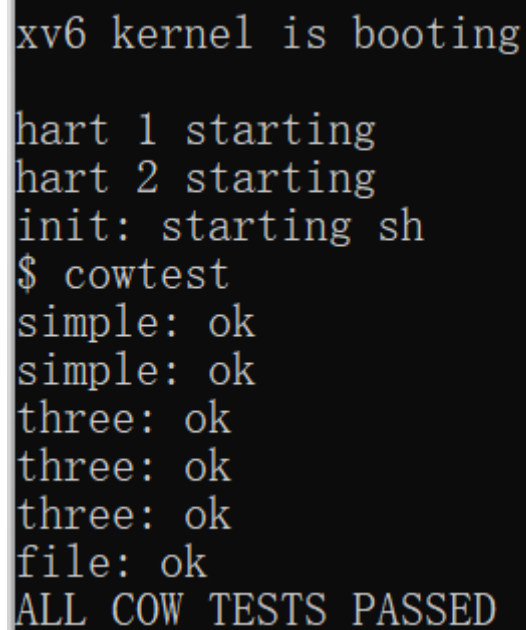
6. 修改 copyout，内核写用户页兼容 COW

C

```
while(len > 0){
    va0 = PGROUNDDOWN(dstva);
    pte_t *pte = walk(pagetable, va0, 0);
    if(*pte & PTE_COW){
        if(cowfault(pagetable, va0) < 0)
            return -1;
    }
    uint64 pa0 = PTE2PA(*pte);
    // 后续内存拷贝逻辑不变
}
```

7. 测试结果:

- 编译运行后输入 `cowtest` 进行测试



```
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
```

- 运行 `usertests` 进行测试

```
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$
```

三、实验中遇到的问题和解决办法

1. 问题：多次 fork、exit 后内核 panic，kfree 重复释放物理页

原因：uvmcopy 共享页面时未调用 krefinc 增加引用计数，计数减为负数，空闲链表重复插入同一块物理内存。

解决：每次父子共享物理页时加锁执行 krefinc，保证每多一个引用计数同步加一，kfree 仅计数归零才回收页面。

四、实验心得

通过本次写时复制实验，我对于操作系统惰性内存优化思想有了更深的理解。如果想要降低 fork 创建子进程的内存与 CPU 开销，这可能涉及到页表共享、物理页引用计数、缺页异常捕获等底层机制，从而也加深了对虚拟内存、进程资源隔离的了解。此外这次实验也涉及用户态、内核态多模块联动修改，我再次巩固了页表项标志扩展、陷阱中断处理、内核与用户内存交互的实现方式。除此之外，通过多组测试用例反复调试，我还明白，在大规模修改内核内存管理逻辑的时候，应当保证各子系统代码同步适配，一处逻辑遗漏就会出现重复释放、内存泄漏、进程数据污染等严重故障，严重破坏系统稳定性。实现 COW 机制时，不仅要完成页面共享与按需拷贝核心功能，还要兼顾多核锁保护、内存分配失败兜底、只读页面区分等边界场景，分层执行专项

测试与全量系统测试，才能保证新增优化逻辑不会干扰原有进程、文件、内存调度功能，避免内核 panic 或程序异常退出。

Lab6: Multithreading

开始之前，切换到 thread 分支。

```
PowerShell
```

```
git fetch
```

```
git checkout thread
```

```
make clean
```

Uthread: switching between threads (moderate)

一、实验目的

本实验实现 xv6 用户态线程库的上下文切换机制：在 `thread_create` 中为新线程准备独立栈与入口，在 `thread_schedule` / `thread_switch` 中保存/恢复被调用者保存寄存器，使多个用户线程可通过 `thread_yield` 协作式轮转，最终让三个测试线程交错打印并正常退出。

二、实验步骤

1. 在 `struct thread` 中增加上下文保存区

切换时只需保存 callee-saved 寄存器及 `ra/sp`（调用约定保证 caller-saved 已由编译

器处理), 布局与内核 `switch` 一致

C

关键代码 (user/uthread.c)

```
struct thread_context {
    uint64 ra, sp;
    uint64 s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;
};

struct thread {
    char stack[STACK_SIZE];
    int state;
    struct thread_context context;
};
```

2. 完善 `thread_create`: 设置首次运行的 `ra` 与 `sp`

新线程第一次被调度时, `thread_switch` 的 `ret` 跳到 `ra`, 故令 `ra = func`; `sp` 指向该线程栈顶, 保证在自有栈上执行

C

关键代码

```
t->state = RUNNABLE;
t->context.ra = (uint64)func;
t->context.sp = (uint64)t->stack + STACK_SIZE;
```

3. 在 `thread_schedule` 中调用 `thread_switch`

找到下一个 `RUNNABLE` 线程后, 从当前线程 `t` 切换到 `next_thread`, 传入双方 `context` 地址

C

关键代码

```
thread_switch((uint64)&t->context, (uint64)&next_thread->context);
```

4. 实现汇编 thread_switch

先把旧线程寄存器存入 a0 指向的 context，再从 a1 恢复新线程寄存器，最后 ret 回到新线程上次让出点（或首次进入的 func）

Assembly language

关键代码 (user/uthread_switch.S)

thread_switch:

```
sd ra, 0(a0)
```

```
sd sp, 8(a0)
```

```
/* ... sd s0-s11 ... */
```

```
ld ra, 0(a1)
```

```
ld sp, 8(a1)
```

```
/* ... ld s0-s11 ... */
```

```
ret
```

5. 测试结果

- 编译运行后输入 uthread 测试:

```
thread_a 98
thread_b 98
thread_c 99
thread_a 99
thread_b 99
thread_c: exit after 100
thread_a: exit after 100
thread_b: exit after 100
thread_schedule: no runnable threads
```

三、实验遇到的问题和解决办法

无

四、实验心得

用户级线程把“调度”放进了进程内部：内核仍只看到一个进程，真正的轮转由 `thread_yield` → `thread_schedule` → `thread_switch` 完成。关键在于第一次运行与再次恢复要统一——都靠 context 里的 `ra/sp`。对照内核 `swtch` 后，用户态切换的本质也清楚了：保存现场、切换栈、恢复现场并返回。同时我也理解了协作式线程的局限：若某线程阻塞在系统调用里，整个进程都会停住，用户级调度器无法介入。

Using threads (moderate)

一、实验目的

本实验在 Linux 上使用 pthread 改造非线程安全的哈希表：分析多线程 `put` 丢失键的原因，用锁保证正确性，再通过每桶一把锁在正确前提下获得并行加速，使两线程 `puts/s` 至少达到单线程的 1.25 倍，并通过 `ph_safe` 与 `ph_fast`。

二、实验步骤

1. 分析并记录丢键的竞态 (`answers-thread.txt`)

两线程对同一桶头插时，都读到旧 `table[i]`，后写者覆盖先写者，导致结点丢失

Plain Text

关键说明

T1、T2 均见 `table[i] == NULL` 并决定插入

T1: `table[i] = e1`

T2: `table[i] = e2` (`e2->next` 仍为 `NULL`) → `e1/k1` 丢失

2. 为每个哈希桶增加互斥锁并初始化

不同桶无共享链表结点，可并行；同桶的查找与插入需串行

C

关键代码 (`notxv6/ph.c`)

```
pthread_mutex_t locks[NBUCKET];
```

// main 中:

```
for (int i = 0; i < NBUCKET; i++)
```

```
    pthread_mutex_init(&locks[i], NULL);
```

3. 在 `put` / `get` 中按桶加锁

锁住 `key % NBUCKET` 对应桶，覆盖“查找是否存在 + 插入/更新”整个临界区；`get` 同样加锁，避免与并发 `put` 交错

C

关键代码

```
void put(int key, int value) {
```

```
    int i = key % NBUCKET;
```

```
    pthread_mutex_lock(&locks[i]);
```

```
    /* 原查找/更新/insert 逻辑 */
```

```
    pthread_mutex_unlock(&locks[i]);
```

```
}
```

4. 测试结果

在修改代码后重新运行实验程序，验证证在多线程情况下是否解决了缺少键的问题：

```
comeat@wdmzhjhj:~/xv6-labs-2021$ make ph
make: 'ph' is up to date.
comeat@wdmzhjhj:~/xv6-labs-2021$ ./ph 1
100000 puts, 6.455 seconds, 15491 puts/second
0: 0 keys missing
100000 gets, 3.457 seconds, 28923 gets/second
comeat@wdmzhjhj:~/xv6-labs-2021$ ./ph 2
100000 puts, 4.182 seconds, 23914 puts/second
1: 0 keys missing
0: 0 keys missing
200000 gets, 5.872 seconds, 34060 gets/second
```

运行 `./ph 1` 作为单线程基准测试，无并发竞争，不存在键丢失；运行 `./ph 2` 启动两个并发线程，结果先输出 `1: 0 keys missing`、后输出 `0: 0 keys missing`。这是操作系统调度器调度不确定性导致：线程创建顺序不代表线程执行结束顺序，线程 1 先完成测试逻辑并打印统计信息，线程 0 随后完成。两条输出丢失键数量均为 0，证明哈希表同步锁能够正确防止并发修改引发的数据竞争，保证并发读写的正确性。

三、实验遇到的问题和解决办法

无

四、实验心得

这个实验把“正确性”和“可扩展性”分开来看：先加锁消除 lost update，再用更细粒度的每桶锁恢复并行度。哈希表天然适合按桶分区，因为不同桶几乎无共享写入集合。从实践中我认识到：锁的粒度直接影响加速比，粗锁易写但对多核不友好。

Barrier(moderate)

一、实验目的

本实验使用 pthread 互斥锁与条件变量实现屏障 (barrier)：参与线程调用 barrier() 时必须等到全部 nthread 个线程都到达同一轮，才一起离开并进入下一轮，从而消除“有的线程已离开屏障、有的尚未到达”导致的断言失败，并通过多线程反复进入屏障的测试。

二、实验步骤

1. 理解失败原因与 bstate 字段含义

原 barrier() 为空，线程立即返回，assert(i == bstate.round) 失败。需要用 nthread 计数本轮到达数，用 round 标识轮次。

2. 实现 barrier()：最后到达者广播，其余等待

加锁后令 nthread++; 若已到齐则 round++、清零计数并 broadcast；否则在 round 未变时 cond_wait。用局部变量保存进入时的 round，防止快线程进入下一轮时误唤醒逻辑出错。

C

关键代码 (notxv6/barrier.c)

```
pthread_mutex_lock(&bstate.barrier_mutex);
int local_round = bstate.round;
bstate.nthread++;
if(bstate.nthread == nthread){
    bstate.round++;
```

```
    bstate.nthread = 0;
    pthread_cond_broadcast(&bstate.barrier_cond);
} else {
    while(bstate.round == local_round)
        pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
}
pthread_mutex_unlock(&bstate.barrier_mutex);
```

3. 测试结果

- 输入:

```
C
make barrier
./barrier 2
./barrier 4
./barrier 8
```

```
comeat@wdmzhhj:~/xv6-labs-2021$ make barrier
make: 'barrier' is up to date.
comeat@wdmzhhj:~/xv6-labs-2021$ ./barrier 2
OK: passed
comeat@wdmzhhj:~/xv6-labs-2021$ ./barrier 4
OK: passed
comeat@wdmzhhj:~/xv6-labs-2021$ ./barrier 8
OK: passed
```

输出 OK: passed, 证明当前屏障实现满足同步语义: 所有线程到达屏障点后才能统一继续执行, 不存在线程提前越过屏障的竞争错误。为充分验证代码鲁棒性, 使用更多线程 (4、8 线程) 进行压力测试, 进一步确认了计数器、互斥锁与条件变量的配合逻辑在高并发场景下依然可靠。

- 输入 make grade 测试:

```

== Test uthread ==
$ make qemu-gdb
uthread: OK (3.3s)
== Test answers-thread.txt == answers-thread.txt: OK
== Test ph_safe == make[1]: Entering directory '/home/comeat/xv6-labs-2021'
gcc -o ph -g -O2 -DSOL_THREAD -DLAB_THREAD notxv6/ph.c -pthread
make[1]: Leaving directory '/home/comeat/xv6-labs-2021'
ph_safe: OK (12.5s)
== Test ph_fast == make[1]: Entering directory '/home/comeat/xv6-labs-2021'
make[1]: 'ph' is up to date.
make[1]: Leaving directory '/home/comeat/xv6-labs-2021'
ph_fast: OK (26.2s)
== Test barrier == make[1]: Entering directory '/home/comeat/xv6-labs-2021'
gcc -o barrier -g -O2 -DSOL_THREAD -DLAB_THREAD notxv6/barrier.c -pthread
make[1]: Leaving directory '/home/comeat/xv6-labs-2021'
barrier: OK (4.0s)
== Test time ==
time: OK
Score: 60/60

```

三、实验遇到的问题和解决办法

无

四、实验心得

屏障是“所有人到齐再一起走”的同步原语，条件变量提供了比忙等更合适的等待/唤醒手段。本实验的难点不在单次汇合，而在多轮复用：既要重置计数，又要防止快线程干扰慢线程仍在使用的状态。用 `local_round` 与 `round` 比较来判定“这一轮是否结束”，是处理这种世代（generation）问题的常用技巧，和 xv6 里 `sleep/wakeup` 要避免丢失唤醒、以及用条件循环等待的思路是相通的。实现并通过测试后，我对锁、条件变量与多线程协调的关系有了更具体的认识。

Lab7: networking

开始之前，切换到 `net` 分支。

PowerShell


```
git fetch
git checkout net
make clean
```

Your Job (hard)

一、实验目的

本实验基于 QEMU 模拟的 E1000 网卡，完成 xv6 操作系统网卡设备驱动的收发核心逻辑，理解 PCI 设备枚举、DMA 环形描述符队列、内存映射寄存器、硬件中断处理机制，掌握 mbuf 数据包管理、ARP/UDP/IP 简易协议栈联动流程，实现数据包收发与主机网络互通，学会借助抓包工具调试网络报文，完整掌握操作系统硬件网络驱动开发流程。

二、实验步骤

1. 修改文件：kernel/e1000.c，实现 e1000_transmit()

关键代码：

```
C
acquire(&e1000_lock);
i = regs[E1000_TDT];
if((tx_ring[i].status & E1000_TXD_STAT_DD) == 0){
    release(&e1000_lock);
    return -1;
}
if(tx_mbufs[i])
```

```

    mbuffree(tx_mbufs[i]);
tx_mbufs[i] = m;
tx_ring[i].addr = (uint64)m->head;
tx_ring[i].length = m->len;
tx_ring[i].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
__sync_synchronize();
regs[E1000_TDT] = (i + 1) % TX_RING_SIZE;
release(&e1000_lock);
return 0;

```

说明：

- 读 E1000_TDT 得到网卡期望的下一个 TX 描述符下标
- 若该描述符没有 E1000_TXD_STAT_DD，说明环满/上次发送未完成，返回 -1 让调用方释放 mbuf
- 若该槽之前有 mbuf，先 mbuffree（网卡已发完）
- 填入 m->head、m->len；EOP 表示一包结束，RS 表示发完后写回 DD
- 用内存屏障后再推进 TDT，避免网卡读到未写完的描述符
- 加锁：多进程与中断可能同时访问 E1000

2. 实现 e1000_rcv()

关键代码：

```

C
for(;;){
    acquire(&e1000_lock);
    i = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
    if((rx_ring[i].status & E1000_RXD_STAT_DD) == 0){

```

```

    release(&e1000_lock);
    return;
}
m = rx_mbufs[i];
m->len = rx_ring[i].length;
rx_mbufs[i] = mbufalloc(0);
if(!rx_mbufs[i])
    panic("e1000");
rx_ring[i].addr = (uint64)rx_mbufs[i]->head;
rx_ring[i].status = 0;
__sync_synchronize();
regs[E1000_RDT] = i;
release(&e1000_lock);
net_rx(m); // 必须在释放锁之后调用
}

```

说明：

- 下一包在 $(RDT + 1) \% RX_RING_SIZE$ (init 里 $RDT = RX_RING_SIZE - 1$)
- 无 `E1000_RXD_STAT_DD` 则没有新包，退出循环
- 用描述符里的 `length` 设置 `m->len`，把该 mbuf 交给 `net_rx()`
- 再 `mbufalloc` 一块新缓冲填回描述符并清 `status`，供网卡下次 DMA
- 把 `RDT` 更新为刚处理完的下标
- 循环处理，直到环上暂时没有更多包（总包数会超过 ring size）
- 先放锁再 `net_rx`：ARP 应答可能再调 `e1000_transmit`，避免死锁

3. 测试结果

- 运行两个终端，终端 1 运行： `make server`，终端 2 运行： `make qemu`

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次 E1000 网卡驱动实验，我对于硬件网卡收发流程有了更深的理解。如果想要实现操作系统网络数据包收发功能，这可能涉及到 DMA 环形描述符、内存映射寄存器读写、硬件中断响应等底层硬件交互逻辑，从而也加深了对设备驱动、内核网络协议栈协同工作机制的了解。此外这次实验也涉及内核并发锁保护、环形队列循环管理、mbuf 数据包生命周期回收，我再次巩固了中断处理、内核缓冲区内存管理的实现思路。除此之外，通过多轮网络测试与 pcap 报文抓包调试，我还明白，在编写硬件驱动代码的时候，应当严格遵循硬件手册规范操作寄存器与描述符，同时做好并发互斥保护，环形队列边界、数据包释放时机、中断触发条件处理不当都会出现丢包、死锁、内存泄漏、收发无响应等问题，严重影响网络通信稳定性。进行完整单进程、多进程、DNS 全场景网络测试，配合 tcpdump 分析报文内容，才能保证驱动收发逻辑正常，实现虚拟机与主机双向网络通信。

Lab8: locks

开始之前，切换到 lock 分支。

```
PowerShell
git fetch
git checkout lock
make clean
```

Memory allocator (moderate)

一、实验目的

本实验针对原版单一全局 kmem 锁多核争抢严重的问题，设计每 CPU 独立空闲链表，搭配跨 CPU 内存窃取机制，拆分全局锁为多组 CPU 私有细粒度锁，降低 kalloc/kfree 并发自旋次数，掌握多核私有资源管理、中断上下文 CPU 编号读取、跨核资源调度逻辑，通过 kalloc_test 验证锁竞争指标显著下降，保证内存分配释放功能完整可用。

二、实验步骤

1. 将 kmem 修改为数组形式的结构体

```
C
struct {
    struct spinlock lock;
    struct run *freelist;
} kmem[NCPU];
```

2. 修改 kinit () 初始化函数

原版只初始化一把锁、全部内存丢给全局链表；新版要循环初始化所有 CPU 的锁，锁名必须以 kmem 开头（评测脚本强制要求），所有空闲内存全部放入 CPU0 的链表。

```
C
```

```

void
kinit()
{
    // 逐个初始化每个 CPU 的锁
    for(int i = 0; i < NCPU; i++){
        initlock(&kmem[i].lock, "kmem");
    }
    freerange(end, (void*)PHYSTOP);
}

```

3. 修改 kfree () 释放函数

释放页面时，归还当前运行 CPU 自己的空闲链表，不能再丢全局链表。

关键点：调用 cpuid()前必须 push_off() 关中断，否则 CPU 编号会错乱、死锁。

```

C++
void
kfree(void *pa)
{
    struct run *r;
    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");
    r = (struct run*)pa;

    push_off();    // 关中断，安全获取 CPU id
    int cpu = cpuid();
    acquire(&kmem[cpu].lock);
    // 头插法放入当前 CPU 链表
    r->next = kmem[cpu].freelist;

```

```

kmem[cpu].freelist = r;
release(&kmem[cpu].lock);
pop_off();    // 恢复中断
}

```

4. 重写 kalloc () 分配逻辑

- 优先从当前 CPU 本地链表拿页面，有直接返回；
- 本地链表空：遍历所有其他 CPU，偷空闲页面；
- 全部 CPU 都无空闲页，返回 0。

```

C++
void *
kalloc(void)
{
    struct run *r;
    int cpu;

    // 必须先关中断，才能安全调用 cpuid()
    push_off();
    cpu = cpuid();

    // 先从当前 CPU 自己的空闲链表拿页面
    acquire(&kmem[cpu].lock);
    r = kmem[cpu].freelist;
    if(r){
        kmem[cpu].freelist = r->next;
        release(&kmem[cpu].lock);
        pop_off();
    }
}

```



```

// 原有的填充垃圾内存逻辑保留
memset((char*)r, 5, PGSIZE);
return r;
}

release(&kmem[cpu].lock);

// 本地链表空，去偷窃其他 CPU 的页面
for(int i = 0; i < NCPU; i++){
    if(i == cpu) continue; // 跳过自己，不用再查一遍

    acquire(&kmem[i].lock);
    r = kmem[i].freelist;
    if(r){
        kmem[i].freelist = r->next;
        release(&kmem[i].lock);
        pop_off();
        memset((char*)r, 5, PGSIZE);
        return r;
    }
    release(&kmem[i].lock);
}

//所有 CPU 都没有空闲页，分配失败
pop_off();
return 0;
}

```

5. 测试结果：

- 输入 `kalloctest` 测试:

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ kalloctest
start test1
test1 results:
--- lock kmem/bcache stats
lock: kmem: #test-and-set 0 #acquire() 88798
lock: kmem: #test-and-set 0 #acquire() 163144
lock: kmem: #test-and-set 0 #acquire() 181125
lock: bcache: #test-and-set 0 #acquire() 1248
--- top 5 contended locks:
lock: proc: #test-and-set 1514808 #acquire() 8276039
lock: proc: #test-and-set 1494075 #acquire() 8276038
lock: proc: #test-and-set 1283644 #acquire() 8276038
lock: proc: #test-and-set 1282865 #acquire() 8276039
lock: proc: #test-and-set 1273594 #acquire() 8276039
tot= 0
test1 OK
start test2
total free number of pages: 32499 (out of 32768)
.....
test2 OK
```

- 输入 `usertests sbrkmuch` 测试:

```
$ usertests sbrkmuch
usertests starting
test sbrkmuch: OK
ALL TESTS PASSED
```

- 输入 `usertests` 测试:

```
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

三、实验中遇到的问题和解决办法

无

四、实验心得

通过本次实验，我对于多核内存资源并发调度有了更深的理解。如果想要消除单一全局内存锁带来的大量自旋争抢，这可能涉及按 CPU 拆分私有空闲链表、设计跨 CPU 内存窃取逻辑，从而也加深了自旋锁、多核并行资源隔离机制的了解。此外这次实验涉及中断开关控制、当前 CPU 编号获取、多锁初始化命名规范，我再次巩固了内核多核编程基础操作。除此之外，通过 `kalloctest` 压力测试对比优化前后锁 `fetch-and-add` 统计值，我还明白粗粒度全局锁会严重拖慢多核并发性能，拆分私有链表时要处理好本地分配、异地窃取两条路径的锁保护，同时保证内存不会重复释放、不会内存泄漏，充分多进程内存压力测试后，才能确认内存分配器并行优化稳定可靠。

Buffer cache (hard)

一、实验目的

本实验旨在降低 xv6 块缓存 (buffer cache) 在多进程并发访问文件系统时对全局锁 `bcache.lock` 的争用。通过将单一 LRU 链表改造为带每桶独立锁的哈希表，并改用时间戳实现近似 LRU 替换，使不同块的查找与释放尽量互不阻塞，从而在 `bcachetest` 中将 `bcache` 相关锁的空转次数 (`tot`) 降至 500 以下，同时保证缓存中同一磁盘块至多一份副本，并通过 `usertests` 验证正确性。

二、实验步骤

1. 扩展 `struct buf`，支持时间戳与桶归属

去掉全局 LRU 链表后，用最后使用时间选择牺牲页；用 `bucket` 记录 buffer 实际挂

在哪个哈希桶，避免 eviction 时按错误桶 unlink 破坏链表。

C

关键代码 (kernel/buf.h)

```
uint timestamp;
uint bucket;
struct buf *prev;
struct buf *next;
```

2. 将 bcache 改为「全局锁 + 每桶一把锁」的哈希表

用质数个桶（13）按块号哈希分散热点；命中路径只获取对应桶锁；全局

`bcache.lock` 仅用于串行化 miss 时的替换，避免所有访问都挤在一把大锁上。锁名均以 "bcache" 开头，便于评分脚本统计。

C

关键代码 (kernel/bio.c)

```
#define NBUCKET 13
```

```
struct {
    struct spinlock lock;
    struct buf buf[NBUF];
    struct {
        struct spinlock lock;
        struct buf head;
    } bucket[NBUCKET];
} bcache;
```

```
static uint bhash(uint dev, uint blockno) {
    return (dev + blockno) % NBUCKET;
```

```
}
```

3. 重写 binit: 初始化桶锁, 并将全部 buffer 挂入桶 0

每个桶维护双向循环链表; 启动时所有空闲 buffer 先放入 bucket 0, 之后按 (dev, blockno) 迁移到对应桶。

C

关键代码

```
initlock(&bcache.lock, "bcache");
for(int i = 0; i < NBUCKET; i++){
    initlock(&bcache.bucket[i].lock, "bcache.bucket");
    // 初始化 bucket[i].head 双向循环链表
}
// 所有 buf 挂到 bucket[0], bucket=0, refcnt=0, timestamp=0
```

4. 改造 bget 命中路径: 只锁目标桶

先按哈希找到桶并加锁, 链表中匹配 (dev, blockno) 则 refcnt++ 并返回; 不同块落到不同桶时可并行查找, 降低争用。

C

关键代码

```
uint bi = bhash(dev, blockno);
acquire(&bcache.bucket[bi].lock);
for(b = bcache.bucket[bi].head.next; b != &bcache.bucket[bi].head; b = b->next){
    if(b->dev == dev && b->blockno == blockno){
        b->refcnt++;
    }
}
```

```

    release(&bcache.bucket[bi].lock);
    acquiresleep(&b->lock);
    return b;
}
}
release(&bcache.bucket[bi].lock); // miss 时先放桶锁，再拿全局锁，避免死锁

```

5. 改造 bget 未命中 / eviction：只锁受害桶与目标桶

获取 bcache.lock 串行化替换；double-check 防止并发插入重复副本；按时间戳选择 refcnt==0 的 LRU 牺牲页；用 bacquire 按桶编号顺序加锁（1~2 个桶），校验后 unlink、改写身份并插入新桶。切勿一次锁住全部桶，否则会与并发命中路径严重冲突。

C

关键代码

```

acquire(&bcache.lock);
// double-check 目标桶后，选 timestamp 最小的 victim
uint vbi = victim->bucket;
bacquire(vbi, bi);
if(victim->refcnt != 0 || victim->bucket != vbi){ /* 重试 */ }
// unlink → 设置 dev/blockno/valid/refcnt/bucket → insert 到 bucket[bi]
brelease(vbi, bi);
release(&bcache.lock);

```

6. 改造 brelse / bpin / bunpin：仅使用桶锁

释放与引用计数调整不再移动全局 LRU 链表，也不再持有 bcache.lock；refcnt 降为 0 时更新 timestamp，供后续 eviction 使用。

C

关键代码

```
uint bi = bhash(b->dev, b->blockno);
acquire(&bcache.bucket[bi].lock);
b->refcnt--;
if(b->refcnt == 0)
    b->timestamp = ticks;
release(&bcache.bucket[bi].lock);
```

7. 增大文件系统容量并重建镜像

lock 实验中 `bcachetest`、`usertests` 占用块数较多，将 `FSSIZE` 从 1000 调整为 10000，避免磁盘耗尽；修改后执行 `make clean && make` 重建 `fs.img`。

C

关键代码 (kernel/param.h)

```
#define FSSIZE 10000 // size of file system in blocks (lock lab needs more
space)
```

8. 测试结果：

- 输入 `bcachetest` 测试：

```

comeat@wdmzhhj: ~/xv6-labs-2021
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ bcachetest
start test0
test0 results:
--- lock kmem/bcache stats
lock: kmem: #test-and-set 0 #acquire() 32999
lock: kmem: #test-and-set 0 #acquire() 61
lock: kmem: #test-and-set 0 #acquire() 40
lock: bcache: #test-and-set 0 #acquire() 78
lock: bcache.bucket: #test-and-set 0 #acquire() 4158
lock: bcache.bucket: #test-and-set 0 #acquire() 4129
lock: bcache.bucket: #test-and-set 0 #acquire() 2121
lock: bcache.bucket: #test-and-set 0 #acquire() 4286
lock: bcache.bucket: #test-and-set 0 #acquire() 2276
lock: bcache.bucket: #test-and-set 0 #acquire() 4280
lock: bcache.bucket: #test-and-set 0 #acquire() 4323
lock: bcache.bucket: #test-and-set 0 #acquire() 6612
lock: bcache.bucket: #test-and-set 0 #acquire() 6623
lock: bcache.bucket: #test-and-set 0 #acquire() 6937
lock: bcache.bucket: #test-and-set 0 #acquire() 6183
lock: bcache.bucket: #test-and-set 0 #acquire() 6196
lock: bcache.bucket: #test-and-set 0 #acquire() 6190
--- top 5 contended locks:
lock: proc: #test-and-set 460306 #acquire() 1895828
lock: proc: #test-and-set 452760 #acquire() 1875317
lock: proc: #test-and-set 425428 #acquire() 1895830
lock: proc: #test-and-set 416533 #acquire() 1895828
lock: proc: #test-and-set 412700 #acquire() 1895830
tot= 0
test0: OK
start test1
test1 OK

```

- 输入 `usertests` 测试:

```

test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$ _

```

- 输入 `make grade` 测试:


```

== Test running kallocetest ==
$ make qemu-gdb
(45.0s)
== Test kallocetest: test1 ==
kallocetest: test1: OK
== Test kallocetest: test2 ==
kallocetest: test2: OK
== Test kallocetest: sbrkmuch ==
$ make qemu-gdb
kallocetest: sbrkmuch: OK (5.9s)
== Test running bcachetest ==
$ make qemu-gdb
(6.1s)
== Test bcachetest: test0 ==
bcachetest: test0: OK
== Test bcachetest: test1 ==
bcachetest: test1: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (89.5s)
(Old xv6.out.usertests failure log removed)
== Test time ==
time: OK
Score: 70/70

```

三、实验中遇到的问题和解决办法

1. 出现报错: bcachetest: test0: FAIL, tot=1328

- 原因: miss 替换时一次性 acquire 全部 13 个桶锁。test0 工作集大于 NBUF, 并发读会频繁 eviction, 其它 CPU 上的命中/brelse 全部被堵住, 锁空转次数飙升。
- 解决: eviction 只锁受害桶与目标桶, 并用统一顺序加锁避免死锁; 命中与释放路径仍只碰单个桶锁。修改后 tot 降至约 200 余, test0 通过。

2. 出现报错: panic: balloc: out of blocks

```

test0: OK
start test1
panic: balloc: out of blocks

```

- 原因: FSSIZE=1000 偏小; test0 会残留目录与文件, test1 还需再分配约百块数据; 重复测试或未重建的旧 fs.img 更易把空闲块耗尽。
- 解决: 将 FSSIZE 改为 10000, 并 make clean 后重新生成 fs.img 再测; 同时为 buf 增加 bucket 字段, 保证换桶 unlink 正确, 避免链表错误间接破坏位图缓存。

四、实验心得

本次实验让我更具体地体会到「锁的粒度」与性能、正确性之间的权衡：全局一把 `bcache.lock` 实现简单，但在多核并发读文件时会成为明显瓶颈；改成哈希分桶后，不同块的查找可以并行，争用显著下降，但 eviction、双锁顺序、double-check 以及「同一块至多一份缓存」等细节都必须仔细处理，否则会出现死锁、重复缓存甚至文件系统元数据损坏。调试过程中，`tot` 过高指向锁持有范围过大，而 `balloc: out of blocks` 则提醒我不仅要改内核逻辑，还要关注实验环境参数（如 `FSSIZE`）与镜像是否与代码同步重建。整体上，这次实验加深了对高速缓存、并发控制与文件系统块分配之间关系的理解，也说明性能优化必须建立在可验证的正确性之上。

Lab9: file system

开始之前，切换到 `fs` 分支。

```
PowerShell
git fetch
git checkout fs
make clean
```

Large files (moderate)

一、实验目的

本实验通过为 xv6 inode 增加双重间接块 (double-indirect block)，将单个文

件的最大容量从 268 块提升到 65803 块 (11 个直接块 + 256 个一级间接块 + 256×256 个二级间接块)。在不改变磁盘上 inode 结构大小的前提下, 重新组织 `addrs[]` 的含义, 并修改 `bmap()` 与 `itrunc()`, 使大文件能够按需分配与完整释放, 最终通过 `bigfile` 与 `usertests`。

二、实验步骤

1. 调整 inode 地址数组布局 (保持 dinode 大小不变)

原 `addrs[12+1]` 共 13 项; 改为 11 个直接项 + 1 个一级间接 + 1 个双重间接, 仍为 13 项, 不扩大磁盘 inode。

```
C
关键代码 (kernel/fs.h / kernel/file.h)
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define MAXFILE (NDIRECT + NINDIRECT + NINDIRECT*NINDIRECT)
uint addrs[NDIRECT+2]; // 11 + 1 + 1 = 13
```

2. 扩展 `bmap()` 支持双重间接映射

逻辑块号 `bn` 先映射直接区, 再映射一级间接区, 剩余部分用 `bn/NINDIRECT`、`bn%NINDIRECT` 两级索引; 各级地址块均按需 `balloc`, 与原实现一致。

```
C
关键代码 (kernel/fs.c)
if(bn < NINDIRECT * NINDIRECT){
    if((addr = ip->addrs[NDIRECT+1]) == 0)
        ip->addrs[NDIRECT+1] = addr = balloc(ip->dev);
    // 读双重间接块, 取 a[bn/NINDIRECT] 得到一级间接块
```

```
// 再读该间接块，取 a[bn%NINDIRECT] 得到数据块
return addr;
}
```

3. 扩展 itrunc() 释放双重间接树

思路：截断时除直接块与一级间接块外，还需遍历双重间接块中每一个一级间接块，释放其指向的全部数据块，再释放间接块与双重间接块本身。

C

关键代码：

```
if(ip->addrs[NDIRECT+1]){
    bp = bread(ip->dev, ip->addrs[NDIRECT+1]);
    for(j = 0; j < NINDIRECT; j++){
        if(a[j] == 0) continue;
        bp2 = bread(ip->dev, a[j]);
        for(k = 0; k < NINDIRECT; k++){
            if(a2[k]) bfree(ip->dev, a2[k]);
        }
        brelse(bp2);
        bfree(ip->dev, a[j]);
    }
    brelse(bp);
    bfree(ip->dev, ip->addrs[NDIRECT+1]);
    ip->addrs[NDIRECT+1] = 0;
}
```

4. 测试结果：

- 输入 bigfile 测试：

```
xv6 kernel is booting
init: starting sh
$ bigfile
.....
.....
.....
.....
.....
wrote 65803 blocks
bigfile done; ok
```

- 输入 `usertests` 测试

```
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

三、实验遇到的问题和解决办法

无

四、实验心得

通过实现双重间接块，我对文件逻辑块号到磁盘块号的映射有了更清晰的认识：直接块适合小文件，一级间接扩展一个数量级，双重间接则以“间接块的间接块”把最大文件大小提升约 256×256 块。实现时既要算清索引公式，也要保证分配与释放对称，否则会留下无法回收的间接块。另外，磁盘数据结构的兼容性很重要——表面

上只是“少一个直接块、多一个双重间接指针”，实质是在固定大小的 `addrs[]` 上重新解释语义。完成本部分后，`bigfile` 能够写满 65803 块，也验证了整条写路径（`bmap`→`balloc`→日志写回）在大文件场景下是可靠的。

Symbolic links (moderate)

一、实验目的

本实验为 xv6 增加符号链接机制：实现 `symlink(target, path)` 系统调用，在 `path` 处创建指向 `target` 的软链接；并修改 `open`，使其在默认情况下沿符号链接查找目标文件，在指定 `O_NOFOLLOW` 时打开链接本身。同时处理链接链与环路（限制跟随深度），使 `symlinktest` 与 `usertests` 全部通过，从而加深对路径名查找与 `inode` 文件类型扩展的理解。

二、实验步骤

1. 在 Makefile 中向 UPROGS 添加 `$U/_symlinktest`

2. 注册 `symlink` 系统调用与用户态接口

分配系统调用号，接入分发表，并生成用户态桩函数。

C

关键代码

```
// kernel/syscall.h
```

```
#define SYS_symlink 22
```

```
// user/usys.pl
entry("symlink");
// user/user.h
int symlink(const char*, const char*);
```

3. 增加文件类型与打开标志

用新类型标记符号链接 inode；用独立比特位表示“不要跟随链接”，避免与 `O_CREATE/O_TRUNC` 冲突。

```
C
关键代码
// kernel/stat.h
#define T_SYMLINK 4
// kernel/fcntl.h
#define O_NOFOLLOW 0x800
```

4. 实现 `sys_symlink`：创建链接并写入目标路径

思路：用 `create(path, T_SYMLINK, ...)` 分配 inode；目标路径不必已存在；将 `target` 写入该 inode 的数据区（本实现写入 `MAXPATH` 字节）。

```
C
关键代码 (kernel/sysfile.c)
begin_op();
ip = create(path, T_SYMLINK, 0, 0);
writei(ip, 0, (uint64)target, 0, MAXPATH);
iunlockput(ip);
end_op();
```

```
return 0;
```

5. 修改 `sys_open`: 默认跟随符号链接

`namei` 得到 `inode` 后, 若类型为 `T_SYMLINK` 且未设置 `O_NOFOLLOW`, 则读出目标路径再次 `namei`; 最多跟随 10 次, 超出则视为环路失败; 目标不存在也返回失败。
`link/unlink` 等仍操作链接自身, 不跟随。

C

关键代码

```
if(ip->type == T_SYMLINK && (omode & O_NOFOLLOW) == 0){  
    for(int depth = 0; depth < 10; depth++){  
        readi(ip, 0, (uint64)target, 0, MAXPATH);  
        iunlockput(ip);  
        if((ip = namei(target)) == 0) return -1;  
        ilock(ip);  
    }  
    if(ip->type == T_SYMLINK) return -1; // 环路或过深  
}
```

6. 测试结果:

- 输入 `symlinktest` 测试

```
$ symlinktest  
Start: test symlinks  
test symlinks: ok  
Start: test concurrent symlinks  
test concurrent symlinks: ok
```


- 输入 `usertests` 测试

```
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

- 输入 `make grade` 测试

```
== Test running bigfile ==
$ make qemu-gdb
running bigfile: OK (167.9s)
== Test running symlinktest ==
$ make qemu-gdb
(0.7s)
== Test    symlinktest: symlinks ==
symlinktest: symlinks: OK
== Test    symlinktest: concurrent symlinks ==
symlinktest: concurrent symlinks: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (230.5s)
== Test time ==
time: OK
Score: 100/100
```

三、实验遇到的问题与解决方法

无

四、实验心得

符号链接把“目录项直接指向 inode”扩展成了“目录项指向一个保存路径字符串的 inode”，真正的目标文件在 `open` 时才解析，因此可以指向尚未创建的路径，也可以形成链接链甚至环。实现时最需要小心的是跟随策略：默认要像 UNIX 一样透明地打开目标文件，同时又必须提供 `O_NOFOLLOW` 以便查看链接本身；环路则要用

深度上限这种工程上可接受的办法切断。和硬链接相比，软链接不增加目标 inode 的 `nlink`，删除目标后链接会悬空，这一点在测试“删除 a 后再打开 b”中体现得很清楚。通过这个练习，路径查找、`namei/create` 与文件类型分发之间的关系变得更加具体，也理解了为什么内核要把“解析路径”和“打开文件”分成既相关又有策略差异的步骤。

Lab10: mmap

开始之前，切换到 `mmap` 分支。

```
PowerShell
git fetch
git checkout mmap
make clean
```

mmap(hard)

一、实验目的

本次实验旨在向 xv6 操作系统添加 `mmap` 和 `munmap` 系统调用，实现对进程地址空间的详细控制。通过实现这两个系统调用，我们可以实现内存映射文件的功能，包括共享内存、将文件映射到进程地址空间等。这有助于理解虚拟内存管理和页面错误处理的机制。

二、实验步骤

1. 在 Makefile 中添加 `$U/_mmaptest`

2. 添加系统调用：添加有关 `mmap` 和 `munmap` 系统调用的定义声明。包括：

- `kernel/syscall.h`:

```
C
#define SYS_mmap 22 // lab10
#define SYS_munmap 23 // lab10
```

- `kernel/syscall.c`:

```
C
extern uint64 sys_mmap(void); // lab10
extern uint64 sys_munmap(void); // lab10

static uint64 (*syscalls[])(void) = {
...
[SYS_mmap] sys_mmap, // lab10
[SYS_munmap] sys_munmap, // lab10
};
```

- `user/usys.pl`:

```
C
entry("mmap"); // lab10
entry("munmap"); // lab10
```

- `user/user.h`:

```
C
void *mmap(void *, int, int, int, int, int); // lab10
int munmap(void *, int); // lab10
```

3. proc.h

```
Go
// lab10: 新增 VMA 宏与结构体定义

struct vm_area {
    uint64 addr; // 映射起始虚拟地址
    int len;     // 映射字节长度
    int prot;    // 读写权限 PROT_READ / PROT_WRITE
    int flags;   // MAP_SHARED / MAP_PRIVATE
    int offset;  // 文件偏移 (实验固定 0)
    struct file *f; // 映射的文件指针
};
```

4. 在 struct proc 结构体中定义相关字段

```
C
#define NVMA 16 // 最多 16 个 mmap 映射区

struct proc {
    ...
    struct vm_area vma[NVMA]; // lab10
}
```

5. kernel/proc.c → allocproc () 新建进程清空 VMA

找到 `allocproc` 函数内初始化进程字段的代码段，在结构体清零逻辑后增加循环清空所有 VMA：

C

// lab10 清空当前所有 VMA 映射

```
for(int i = 0; i < NVMA; i++){
    p->vma[i].addr = 0;
    p->vma[i].len = 0;
    p->vma[i].f = 0;
    p->vma[i].prot = 0;
    p->vma[i].flags = 0;
    p->vma[i].offset = 0;
}
```

6. kernel/proc.c → exit() 进程退出自动释放全部 mmap 映射

找到 exit(int status) 函数, 在关闭文件 for(fcloseset(&p->ofiles[0])); 之后、释放用户页表 uvmfree(p->pagetable, 0); 之前插入 VMA 回收逻辑:

Go

```
for(int i = 0; i < NVMA; i++){
    struct vm_area *v = &p->vma;
    if(v->len == 0)
        continue;
    int pages = v->len / PGSIZE;
    uvmunmap(p->pagetable, v->addr, pages, 0);
    v->f->ref--;
    v->f = 0;
    v->len = 0;
}
```

7. kernel/sysfile.c 实现 sys_mmap

```
C
#include "vm.h" // lab10 新增, 用于页表相关函数

// lab10 完整 sys_mmap 实现
#define MMAP_START 0x40000000 // mmap 分配虚拟地址起始

uint64
sys_mmap(void)
{
    struct proc *p = myproc();
    uint64 addr;
    int len, prot, flags, fd, offset;

    // 解析 6 个入参
    if(argaddr(0, &addr) < 0) return -1;
    if(argint(1, &len) < 0) return -1;
    if(argint(2, &prot) < 0) return -1;
    if(argint(3, &flags) < 0) return -1;
    if(argint(4, &fd) < 0) return -1;
    if(argint(5, &offset) < 0) return -1;

    // 参数合法性校验 (实验限定条件)
    if(addr != 0) return -1; // 用户必须传 0, 由内核分配 VA
    if(len <= 0) return -1;
    if(offset != 0) return -1; // 实验 offset 固定为 0
```

```

if(flags != MAP_SHARED && flags != MAP_PRIVATE) return -1;
if((prot & ~(PROT_READ | PROT_WRITE)) != 0) return -1;
// 校验 fd 合法
if(fd < 0 || fd >= NOFILE || p->ofile[fd] == 0) return -1;

// 寻找空闲 VMA 槽位
int vma_idx = -1;
for(int i = 0; i < NVMA; i++){
    if(p->vma[i].len == 0){
        vma_idx = i;
        break;
    }
}
if(vma_idx == -1) return -1; // VMA 用尽, 分配失败

// 寻找不重叠的虚拟地址, 从 MMAP_START 向上遍历
uint64 va = MMAP_START;
int ok;
do {
    ok = 1;
    uint64 va_end = va + len;
    for(int i = 0; i < NVMA; i++){
        struct vm_area *v = &p->vma[i];
        if(v->len == 0) continue;
        uint64 v_end = v->addr + v->len;
        // 判断区间重叠
        if(!(va_end <= v->addr || va >= v_end)){
            ok = 0;
            va += PGSIZE;
        }
    }
} while (!ok);

```

```

        break;
    }
}
} while(!ok);

// 填充 VMA, 文件引用+1
struct vm_area *new_vma = &p->vma[vma_idx];
new_vma->addr = va;
new_vma->len = len;
new_vma->prot = prot;
new_vma->flags = flags;
new_vma->offset = offset;
new_vma->f = filedup(p->ofile[fd]);

// 懒分配: 不创建任何页表映射, 仅返回虚拟地址
return va;
}

```

8. usertrap.c

```

Go
void
usertrap(void)
{
...
// 新增 mmap 缺页处理分支
else if(r_scause() == 12 || r_scause() == 13 || r_scause() == 15){

```



```

// 12=读缺页 13=取指缺页 15=写缺页
uint64 va = r_stval(); // 故障虚拟地址
struct vm_area *match_vma = 0;

// 遍历进程 VMA, 匹配故障地址是否属于 mmap 映射
for(int i = 0; i < NVMA; i++){
    struct vm_area *v = &p->vma[i];
    if(v->len == 0) continue;
    if(va >= v->addr && va < v->addr + v->len){
        match_vma = v;
        break;
    }
}

// 不是 mmap 区域, 走 panic 报错
if(match_vma == 0){
    goto bad_fault;
}

// 懒分配: 分配一页物理内存并清零
uint64 pa = kalloc();
if(pa == 0) panic("mmap out of memory");
memset((void*)pa, 0, PGSIZE);

// 计算该页在文件内偏移
uint64 page_off = va - match_vma->addr;
uint64 file_off = match_vma->offset + page_off;

// 锁定 inode, 读取文件一页到物理内存

```

```

ilock(match_vma->f->ip);
readi(match_vma->f->ip, pa, file_off, PGSIZE);
iunlock(match_vma->f->ip);

// 组装页权限：必须带用户位 PTE_U
uint64 perm = PTE_U;
if(match_vma->prot & PROT_READ) perm |= PTE_R;
if(match_vma->prot & PROT_WRITE) perm |= PTE_W;

// 建立虚拟地址与物理页映射
if(mappages(p->pagetable, va, PGSIZE, pa, perm) < 0){
    kfree((void*)pa);
    panic("mmap mappages fail");
}
// 缺页处理完成，直接返回用户态，不执行下面 bad_fault
goto skip_fault_handle;
}
// 新增 mmap 缺页处理分支
else {
bad_fault:
    printf("usertrap(): unexpected scause %p pid=%d\n", r_scause(), p->pid);
    printf("      sepc=%p stval=%p\n", r_sepc(), r_stval());
    p->killed = 1;
}
skip_fault_handle:

if(p->killed)
    exit(-1);

```

```

// give up the CPU if this is a timer interrupt.
if(which_dev == 2)
    yield();

usertrapret();
}

```

9. sys_munmap()

```

Go
uint64
sys_munmap(void)
{
    struct proc *p = myproc();
    uint64 addr;
    int len;

    // 解析两个入参：起始虚拟地址、长度
    if(argaddr(0, &addr) < 0)
        return -1;
    if(argint(1, &len) < 0)
        return -1;

    // 地址必须页对齐，长度大于 0
    if(addr % PGSIZE != 0 || len <= 0)
        return -1;
    uint64 end_va = addr + len;

```

```

// 查找匹配的 VMA 区域
struct vm_area *target_vma = 0;
int target_idx = -1;
for(int i = 0; i < NVMA; i++){
    struct vm_area *v = &p->vma[i];
    if(v->len == 0)
        continue;
    // 匹配起始地址，且解除范围不超过该 VMA 总长度
    if(v->addr == addr && end_va <= v->addr + v->len){
        target_vma = v;
        target_idx = i;
        break;
    }
}
if(target_vma == 0)
    return -1; // 找不到对应映射，返回失败

// MAP_SHARED 页面：循环写回文件（简化，不判断脏位）
if(target_vma->flags == MAP_SHARED){
    uint64 cur = addr;
    while(cur < end_va){
        // 查找当前页的 PTE
        pte_t *pte = walk(p->pagetable, cur, 0);
        if(pte && (*pte & PTE_V)){
            uint64 pa = PTE2PA(*pte);
            uint64 file_off = target_vma->offset + (cur - target_vma);
            ilock(target_vma->f->ip);
            writei(target_vma->f->ip, pa, file_off, PGSIZE);
            iunlock(target_vma->f->ip);
        }
        cur += PGSIZE;
    }
}

```

```

    }
    cur += PGSIZE;
}
}

// 解除页表映射
int page_count = len / PGSIZE;
uvmunmap(p->pagetable, addr, page_count, 0);

// 如果释放的是整个 VMA 区间，清空 V、减少文件引用
if(addr == target_vma->addr && end_va == target_vma->addr +
target_vma->len){
    target_vma->f->ref--;
    target_vma->f = 0;
    target_vma->len = 0;
}

return 0;
}

```

10. 找到 `fork(void)` 函数，在 `np->pagetable = uvmcopy(p->pagetable);` 复制用户页表之一添加 VMA 拷贝逻辑：

```

Go
// lab10 拷贝所有 VMA 子进程
for(int i = 0; i < NVMA; i++){
    struct vm_area *src_vma = &p->vma;
    struct vm_area *dst_vma = &np->vma;

```

```
if(src_vma->len == 0){
    dst_vma->len = 0;
    continue;
}
// 完整复制 VMA 所有字段
*dst_vma = *src_vma;
// 文件引用计数+1
dst_vma->f = filedup(src_vma->f);
}
```

11. 测试结果:

- 输入 mmaptest 测试:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
mmap_test starting
test mmap f
test mmap f: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test mmap two files
test mmap two files: OK
mmap_test: ALL OK
fork_test starting
fork_test OK
mmaptest: all tests succeeded
```

- 输入 `usertests` 测试:

```
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$
```

- 输入 `make grade` 测试:

```
== Test running mmaptest ==
$ make qemu-gdb
(3.4s)
== Test mmaptest: mmap f ==
mmaptest: mmap f: OK
== Test mmaptest: mmap private ==
mmaptest: mmap private: OK
== Test mmaptest: mmap read-only ==
mmaptest: mmap read-only: OK
== Test mmaptest: mmap read/write ==
mmaptest: mmap read/write: OK
== Test mmaptest: mmap dirty ==
mmaptest: mmap dirty: OK
== Test mmaptest: not-mapped unmap ==
mmaptest: not-mapped unmap: OK
== Test mmaptest: two files ==
mmaptest: two files: OK
== Test mmaptest: fork_test ==
mmaptest: fork_test: OK
== Test usertests ==
$ make qemu-gdb

usertests: OK (79.7s)
== Test time ==
time: OK
Score: 140/140
```

三、实验中遇到的问题和解决办法

`sys_mmap` 函数的编写上，我们希望完成文件与进程虚拟地址区间的绑定记录。`vm_area` 结构体用于存储每一块 `mmap` 映射的完整信息，然后我们知道缺页异常处理函数 `usertrap` 用于捕获未分配映射页面的访问。在进程首次访问映射虚拟地址时，

会触发缺页异常，内核需要分配物理内存并读取对应文件数据，但不能仅完成页面加载，我还必须区分 MAP_SHARED 与 MAP_PRIVATE 两种映射标识，它决定解除映射时是否要将脏页面写回磁盘。如果是共享映射，munmap 时必须同步修改落盘，私有映射则无需改动源文件。所以我将它设计成：分配 VMA 空闲槽位并记录地址、长度、文件与权限，访问缺页时懒加载页面，解除映射时根据标志位判断是否执行磁盘回写操作，以此完整实现文件内存映射功能。

四、实验心得

通过本次实验，我对于文件懒加载虚拟内存机制有了更深的理解。如果想要实现文件直接映射到进程虚拟地址空间，这可能涉及 VMA 区域管理、缺页异常懒分配、磁盘文件读写同步等底层逻辑，从而也加深了虚拟地址、物理内存、磁盘文件三者交互机制的了解。此外这次实验完整覆盖系统调用注册、进程 VMA 生命周期管理、fork 资源拷贝、exit 资源回收全流程，我再次巩固 RISC-V 各类缺页异常捕获、页表映射构建与销毁的实现方式。除此之外，通过 mmaptest 多组场景反复调试，我还明白在新增用户态内存映射功能时，必须完整处理进程创建、分支、退出全生命周期资源管理，VMA 拷贝、文件引用计数增减、共享页面脏数据回写任何一处遗漏都会造成内存泄漏、文件数据丢失、缺页 panic 等稳定性问题。区分私有映射与共享映射读写逻辑，对缺页、munmap、fork、exit 分别做适配处理，分层执行单项功能测试与全量 usertests 综合测试，才能保证 mmap 映射读写、进程派生、资源释放全部逻辑稳定可用。